

UNIFEOB
CENTRO UNIVERSITÁRIO DA FUNDAÇÃO DE ENSINO OCTÁVIO BASTOS
ESCOLA DE NEGÓCIOS

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
CIÊNCIAS DA COMPUTAÇÃO

**CRUD de Ferramentas e Testes da API no Insomnia – Entrega Prof. Nivaldo
Andrade 15/09/2026**

ESTUDANTES:

25000151 - Guilherme P. da Rosa Santi
25001176 - Henrique de Oliveira Molinari
25000019 - João Augusto de Freitas
25000795 - Kauan Leander Leandrini

São João da Boa Vista–SP
2026

1. Contexto e objetivo

Esta entrega documenta os testes do CRUD de ferramentas da API do SOUFER Tools, feitos no Insomnia. O CRUD é a primeira entidade completa do back-end: cadastrar, listar, consultar, atualizar, marcar etiqueta, disponibilizar e dar baixa em uma ferramenta do almoxarife da Soufer.

O objetivo é provar, rota por rota, que a API responde como o projeto definiu: o caminho feliz (sucesso) e também os caminhos de erro (autenticação, validação, recurso inexistente e regras de negócio). Para isso foram criadas 40 requisições organizadas em 7 pastas, cada uma com testes automáticos, e este documento explica o que cada requisição faz, por que existe e o que mostram os dois prints registrados para ela.

Os arquivos que sustentam esta entrega estão versionados no repositório do projeto (branch *docs/api-07-testes-insomnia-ferramentas*, PR #142):

- *docs/insomnia/soufer-tools-ferramentas.json*: coleção do Insomnia (40 requisições, com documentação e testes dentro de cada uma);
- *docs/insomnia/teste-api-crud-ferramentas.md*: tutorial passo a passo para rodar a coleção;
- *api-07-put-para-patch.md*: registro da troca de *PUT* por *PATCH* na atualização de ferramentas.

2. Ambiente e forma de execução

2.1 Ambiente

Os testes rodaram em ambiente local, contra o banco de desenvolvimento, com a API Node.js/Express (TypeScript) na porta 3000 e o PostgreSQL com o banco *soufer_dev* populado pelo seed (50 ferramentas, colaboradores e empréstimos de exemplo). O cliente HTTP é o Insomnia 13.1.

```
cd api
npm run db:migrate # cria as tabelas
npm run db:seed    # popula usuários, colaboradores e ferramentas
npm run dev        # API em http://localhost:3000
```

Credenciais de teste criadas pelo seed: almoxarife *almoxarife@soufer.com.br / 123456* e perfil consulta pela matrícula *MAT001*.

2.2 Variáveis de ambiente

A coleção não copia token nem ids à mão. Cada requisição que produz um dado o grava em uma variável do Base Environment por meio de um script pós-resposta (aba Scripts, sub-aba After-response), e as requisições seguintes o leem. É o que faz o Bearer Token e as URLs aparecerem como *_.base_url*, *_.ferramenta_id* etc. em roxo nos prints.

Variável	Quem grava	Quem usa
<i>base_url</i>	já vem preenchida	todas (endereço da API, <i>http://localhost:3000/v1</i>)
<i>token</i>	01 Login do almoxarife	todas as rotas de ferramentas
<i>token_consulta</i>	02 Login do perfil consulta	19 (teste de 403)
<i>grupo_id</i>	03 Obter um grupoid válido	04 Cadastrar

Variável	Quem grava	Quem usa
<i>ferramenta_id</i>	04 Cadastrar	07, 09 a 12, 24, 37 a 39
<i>ferramenta_codigo</i>	04 Cadastrar	08
<i>ferramenta_emprestada_id</i>	13 (auxiliar)	14
<i>ferramenta_indisponivel_id</i>	15 (auxiliar)	16

2.3 Como ler os prints

Cada requisição tem dois prints, sempre na mesma ordem, e todos têm a mesma organização de tela:

- **Print 1 (requisição e resposta):** à esquerda, a requisição configurada (método, URL, e a aba Body ou Params). À direita, a resposta da API: código HTTP, tempo, tamanho e o JSON retornado na aba Preview.
- **Print 2 (script e testes):** à esquerda, a aba Scripts com o script After-response que valida a resposta. À direita, a aba Tests, com um selo PASS para cada teste aprovado.
- Na barra superior aparece o caminho da requisição (coleção, pasta e nome). O número ao lado de Tests (por exemplo, 2/2) mostra quantos testes passaram.
- Nas abas Body, Auth, Headers, Scripts e Docs, o ponto verde indica que há conteúdo configurado. Na requisição 17 a aba Auth não tem ponto verde, porque o teste é justamente sem token.

Observação: o Insomnia exibe os nomes dos testes com a inicial de cada palavra em maiúscula (por exemplo, “Resposta Traz Um Token”), mesmo que no script estejam escritos em frase normal.

3. Rotas testadas

Além do login (necessário para obter o token), as requisições exercitam todas as rotas de */v1/ferramentas* implementadas nas issues API-05 e API-07:

Método	Rota	Função
POST	<i>/v1/auth/Login</i>	Login do almoxarife (e-mail e senha), devolve token JWT de 8h
POST	<i>/v1/consulta/sessao</i>	Sessão do perfil consulta (matrícula), token de 15 minutos
GET	<i>/v1/ferramentas</i>	Lista ferramentas ativas, com paginação, busca e filtros
POST	<i>/v1/ferramentas</i>	Cadastra uma ferramenta (o código de identificação é gerado pelo banco)
GET	<i>/v1/ferramentas/:id</i>	Busca uma ferramenta pelo ID
GET	<i>/v1/ferramentas/por-codigo/:codigo</i>	Busca pelo código de identificação (1 a 9999)
GET	<i>/v1/ferramentas/:id/historico</i>	Histórico de empréstimos e ocorrências
PATCH	<i>/v1/ferramentas/:id</i>	Atualização parcial dos campos editáveis
PATCH	<i>/v1/ferramentas/:id/etiqueta-impressa</i>	Registra a data de impressão da etiqueta
PATCH	<i>/v1/ferramentas/:id/disponibilizar</i>	Tira a ferramenta de “indisponível” após reparo

Método	Rota	Função
DELETE	/v1/ferramentas/:id	Baixa lógica (<i>ativo = false</i>), nunca apaga o registro

Todas as rotas de ferramentas exigem o perfil *almoxarife* e seguem o envelope padrão do projeto: { *data*, *meta* } no sucesso e { *error*: { *code*, *message*, *details* } } no erro.

4. Decisões e convenções

- **PATCH na atualização parcial:** a rota de atualização era *PUT*, mas sempre alterou apenas os campos enviados. Pela convenção REST isso é *PATCH*, e a API foi ajustada. Agora *PATCH /v1/ferramentas/:id* atualiza e o *PUT* responde 404. As requisições 10, 24 e 34 já usam *PATCH*.
- **Códigos HTTP:** 401 quando não há token ou ele é inválido, 403 quando o perfil não tem permissão, 400 para dados inválidos, 404 quando o recurso não existe e 409 para conflito de estado (regras de negócio).
- **Baixa lógica:** o *DELETE* marca a ferramenta como inativa em vez de apagá-la, preservando o histórico. A resposta é 200 com a ferramenta baixada.
- **Pontos que diferem da convenção REST usual e foram mantidos:** o *DELETE* responde 200 (o usual seria 204) e uma referência inexistente (*grupoId* que não existe) responde 400 *FOREIGN_KEY_VIOLATION* (o usual seria 422 ou 409). Os testes seguem o comportamento real da API.
- **Dados do seed:** a requisição 16 (disponibilizar) altera uma ferramenta do seed, e cada rodada cria e dá baixa numa ferramenta de teste. Por isso os testes só devem rodar no banco de desenvolvimento.

5. Resumo dos resultados

As 40 requisições retornaram o status esperado e os 85 testes automáticos foram aprovados (selo PASS em todos). Na tabela, verde indica requisição de sucesso e vermelho indica erro esperado, que é justamente o comportamento que se queria comprovar.

Nº	Rota	Cenário	Resultado	Testes
01	POST /v1/auth/Login	Login do almoxarife	☑ 200, token emitido	3/3
02	POST /v1/consulta/sessao	Login do perfil consulta	☑ 200, token de 15min	2/2
03	GET /v1/ferramentas	Obter um grupold válido	☑ 200, grupo_id obtido	2/2
04	POST /v1/ferramentas	Cadastrar ferramenta	☑ 201, ferramenta criada	3/3
05	GET /v1/ferramentas	Listar ferramentas (paginado)	☑ 200, lista paginada	3/3

Nº	Rota	Cenário	Resultado	Testes
06	<i>GET /v1/ferramentas</i>	Listar com filtros, busca e ordenação	☑ 200, lista filtrada	3/3
07	<i>GET /v1/ferramentas/:id</i>	Buscar ferramenta por ID	☑ 200, dados da ferramenta	2/2
08	<i>GET /v1/ferramentas/por-codigo/:codigo</i>	Buscar ferramenta por código de identificação	☑ 200, dados da ferramenta	2/2
09	<i>GET /v1/ferramentas/:id/historico</i>	Consultar histórico da ferramenta	☑ 200, histórico	2/2
10	<i>PATCH /v1/ferramentas/:id</i>	Atualizar ferramenta (PATCH, campos parciais)	☑ 200, campos alterados	3/3
11	<i>PATCH /v1/ferramentas/:id/etiqueta-impressa</i>	Marcar etiqueta como impressa	☑ 200, etiqueta registrada	2/2
12	<i>PATCH /v1/ferramentas/:id/disponibilizar</i>	Disponibilizar ferramenta que já está disponível	✗ 409 FERRAMENTA_JA_DISPONIVEL	2/2
13	<i>GET /v1/ferramentas</i>	(auxiliar) Localizar ferramenta com empréstimo aberto	☑ 200, ferramenta em uso	2/2
14	<i>DELETE /v1/ferramentas/:id</i>	Dar baixa em ferramenta com empréstimo aberto	✗ 409 FERRAMENTA_COM_EMPRESTIMO_ABERTO	2/2
15	<i>GET /v1/ferramentas</i>	(auxiliar) Localizar ferramenta indisponível	☑ 200, ferramenta indisponível	2/2
16	<i>PATCH /v1/ferramentas/:id/disponibilizar</i>	Disponibilizar ferramenta após reparo	☑ 200, volta a disponível	2/2
17	<i>GET /v1/ferramentas</i>	Listar sem token	✗ 401 TOKEN_NOT_PROVIDED	2/2
18	<i>GET /v1/ferramentas</i>	Listar com token inválido	✗ 401 TOKEN_INVALID	2/2
19	<i>GET /v1/ferramentas</i>	Listar com perfil consulta	✗ 403 ACCESS_DENIED	2/2
20	<i>POST /v1/ferramentas</i>	Cadastrar sem nome	✗ 400 VALIDATION_ERROR	2/2

Nº	Rota	Cenário	Resultado	Testes
21	<i>POST /v1/ferramentas</i>	Cadastrar com nome de 1 caractere	X 400 VALIDATION_ERROR	2/2
22	<i>POST /v1/ferramentas</i>	Cadastrar sem grupold	X 400 VALIDATION_ERROR	2/2
23	<i>POST /v1/ferramentas</i>	Cadastrar com grupold inexistente	X 400 FOREIGN_KEY_VIOLATION	2/2
24	<i>PATCH /v1/ferramentas/:id</i>	Atualizar com corpo vazio	X 400 VALIDATION_ERROR	2/2
25	<i>GET /v1/ferramentas/abc</i>	Buscar com ID não numérico	X 400 VALIDATION_ERROR	2/2
26	<i>GET /v1/ferramentas/-1</i>	Buscar com ID negativo	X 400 VALIDATION_ERROR	2/2
27	<i>GET /v1/ferramentas</i>	Listar com status inválido	X 400 VALIDATION_ERROR	2/2
28	<i>GET /v1/ferramentas</i>	Listar com limit acima do máximo	X 400 VALIDATION_ERROR	2/2
29	<i>GET /v1/ferramentas/por-codigo/0</i>	Buscar por código 0	X 400 VALIDATION_ERROR	2/2
30	<i>GET /v1/ferramentas/por-codigo/10000</i>	Buscar por código 10000	X 400 VALIDATION_ERROR	2/2
31	<i>GET /v1/ferramentas/999999</i>	Buscar ID inexistente	X 404 FERRAMENTA_NOT_FOUND	2/2
32	<i>GET /v1/ferramentas/por-codigo/9999</i>	Buscar código inexistente	X 404 FERRAMENTA_NOT_FOUND	2/2
33	<i>GET /v1/ferramentas/999999/historico</i>	Histórico de ID inexistente	X 404 FERRAMENTA_NOT_FOUND	2/2
34	<i>PATCH /v1/ferramentas/999999</i>	Atualizar ID inexistente	X 404 FERRAMENTA_NOT_FOUND	2/2
35	<i>PATCH /v1/ferramentas/999999/etiqueta-impressa</i>	Marcar etiqueta de ID inexistente	X 404 FERRAMENTA_NOT_FOUND	2/2
36	<i>PATCH /v1/ferramentas/999999/disponibilizar</i>	Disponibilizar ID inexistente	X 404 FERRAMENTA_NOT_FOUND	2/2
37	<i>DELETE /v1/ferramentas/:id</i>	Dar baixa na ferramenta criada	☑ 200, baixa lógica	2/2
38	<i>GET /v1/ferramentas/:id</i>	Buscar ferramenta já baixada	X 404 FERRAMENTA_NOT_FOUND	2/2
39	<i>DELETE /v1/ferramentas/:id</i>	Dar baixa duas vezes	X 404 FERRAMENTA_NOT_FOUND	2/2

Nº	Rota	Cenário	Resultado	Testes
40	<i>DELETE /v1/ferramentas/999999</i>	Dar baixa em ID inexistente	X 404 FERRAMENTA_NOT_FOUND	2/2

6. Detalhamento das requisições

Cada requisição a seguir traz: o que faz, por que foi criada, como está configurada, o resultado esperado e a explicação dos dois prints. O texto de cada requisição vem da documentação escrita dentro do próprio Insomnia (aba Docs).

6.1 Autenticação e preparação

Requisições que precisam rodar primeiro. Obtêm os tokens e o *grupoId* que as demais usam, gravando-os em variáveis de ambiente. Nada aqui altera o banco.

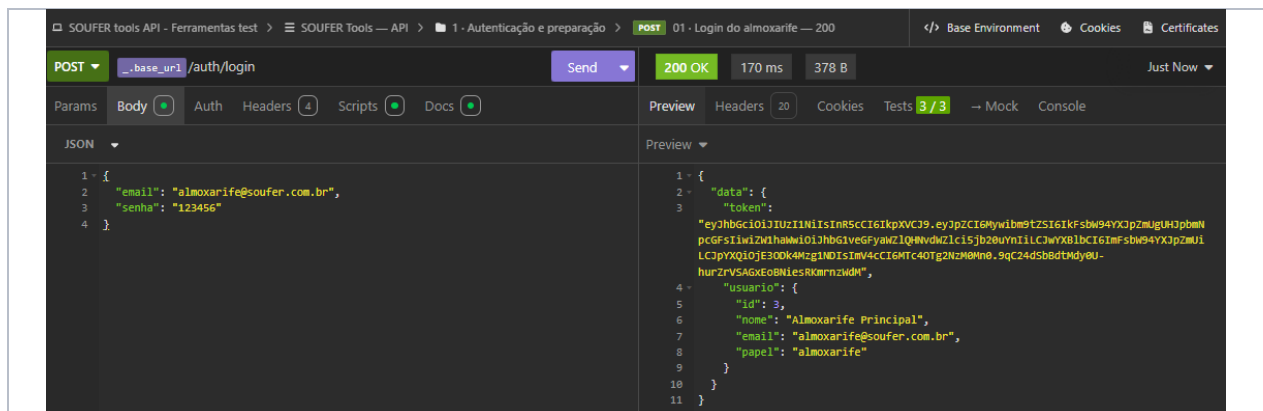
Requisição 01 · Login do almoxarife (200)

O que faz. Autentica o almoxarife e devolve um token JWT válido por 8 horas.

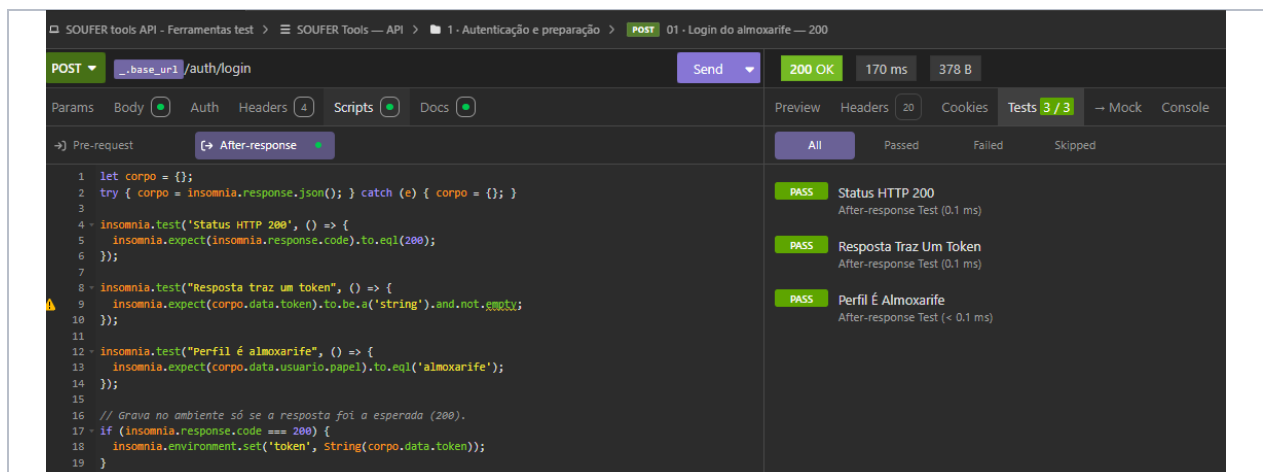
Por que existe. Todas as rotas de */ferramentas* exigem perfil *almojarife*. Sem este login nenhuma outra requisição funciona. É a primeira a ser executada.

Método e rota	POST /v1/auth/Login
Autenticação	Nenhuma (sem cabeçalho Authorization)
Corpo (JSON)	{ "email": "almoxarife@soufer.com.br", "senha": "123456" }
Grava no ambiente	token
Resultado esperado	HTTP 200 com <i>data.token</i> e <i>data.usuario</i> (perfil <i>almoxarife</i>).

Print 1 · Requisição e resposta. Aba Body da requisição *POST /auth/Login* com o JSON enviado (e-mail e senha do almoxarife do seed). À direita, a resposta **200 OK** em 170 ms: *data.token* traz o JWT e *data.usuario* identifica quem entrou (id 3, “Almoxarife Principal”, papel *almoxarife*). O selo Tests já indica 3/3.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta e grava *token*, e aba Tests com 3/3 testes aprovados (PASS): “Status HTTP 200”; “Resposta traz um token”; “Perfil é almoxarife”.



Observações. O token dura 8 horas. Se as demais requisições passarem a retornar 401, rode esta de novo. Requer que o seed (`npm run db:seed`) tenha sido executado.

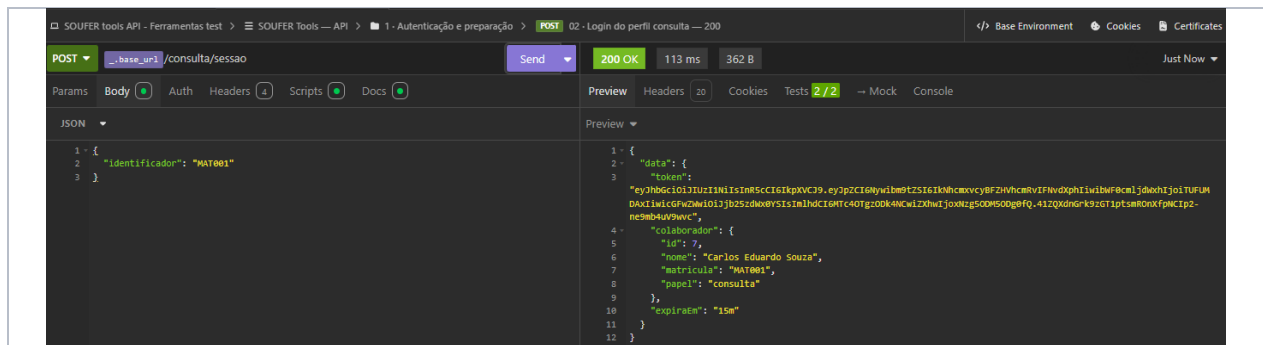
Requisição 02 · Login do perfil consulta (200)

O que faz. Abre uma sessão de consulta (somente leitura) para um colaborador, identificado pela matrícula.

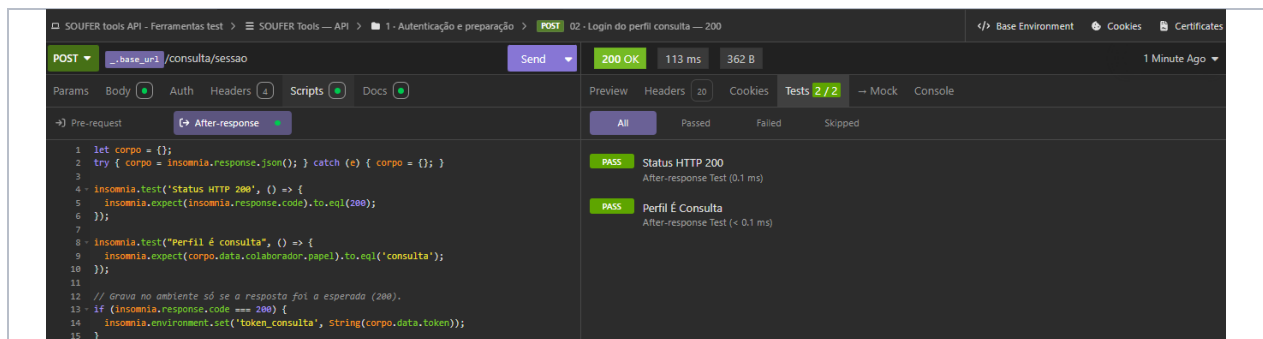
Por que existe. Precisamos de um token de perfil diferente do almoxarife para provar que a API bloqueia esse perfil nas rotas de ferramentas (teste de HTTP 403, requisição 19).

Método e rota	<i>POST /v1/consulta/sessao</i>
Autenticação	Nenhuma (sem cabeçalho Authorization)
Corpo (JSON)	<pre>{ "identificador": "MAT001" }</pre>
Grava no ambiente	<i>token_consulta</i>
Resultado esperado	HTTP 200 com <i>data.token</i> (validade de 15 minutos) e os dados do colaborador.

Print 1 · Requisição e resposta. Aba Body com { "identificador": "MAT001" }, a matrícula do colaborador. A resposta **200 OK** (113 ms) devolve o token, o colaborador (id 7, Carlos Eduardo Souza, papel **consulta**) e **expiraEm: "15m"**, confirmando que a sessão de consulta dura 15 minutos.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta e grava *token_consulta*, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 200”; “Perfil é consulta”.



Observações. A sessão de consulta expira em 15 minutos. Se o teste 403 falhar por token expirado, rode esta requisição de novo.

Requisição 03 · Obter um grupoid válido (200)

O que faz. Lista apenas 1 ferramenta e guarda o *grupo_id* dela.

Por que existe. Para cadastrar uma ferramenta (requisição 04) é obrigatório informar um *grupoId* que exista. Os ids dos grupos mudam conforme quantas vezes o seed foi executado em cada máquina, então não dá para fixar um número. Não existe rota de listagem de grupos, por isso aproveitamos o *grupo_id* de uma ferramenta já cadastrada.

Método e rota	GET /v1/ferramentas
Query	Limit=1
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Grava no ambiente	<i>grupo_id</i>
Resultado esperado	HTTP 200 com uma ferramenta e <i>meta.total</i> maior que zero.

Print 1 · Requisição e resposta. Requisição *GET /ferramentas?Limit=1*. A aba Body mostra “No Body”, porque um GET não envia corpo. A resposta **200 OK** (114 ms) traz uma única ferramenta (id 48, Alicates de Solda a Ponto Lincoln Electric, *grupo_id* 7) e *meta.total* = 52. É desse *grupo_id* 7 que sai o valor usado no cadastro.

The screenshot shows the SOUFER tools API interface. The URL bar displays `GET /base_url/ferramentas?limit=1`. The response status is **200 OK** with a response time of 114 ms and a body size of 484 B. The response body is a JSON object:

```
1 {
2   "data": [
3     {
4       "id": 48,
5       "nome": "Alicate de Solda a Ponto Lincoln Electric",
6       "descricao": "Alicate para solda a ponto de chapas finas",
7       "marca": "Lincoln Electric",
8       "modelo": "SP-200",
9       "codigo_identificacao": 45,
10      "grupo_id": 7,
11      "subgrupo_id": null,
12      "setor_id": 8,
13      "localizacao": "Bancada Solda 03",
14      "status": "disponivel",
15      "motivo_indisponivel": null,
16      "etiqueta_impressao": "2026-09-16T03:28:51.634Z",
17      "ativo": true,
18      "created_at": "2026-09-16T01:06:55.547Z"
19    }
20  ],
21   "meta": {
22     "page": 1,
23     "limit": 1,
24     "total": 52,
25     "totalPages": 52
26   }
27 }
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta e grava *grupo_id*, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 200”; “Existe ao menos uma ferramenta cadastrada”.

The screenshot shows the SOUFER tools API interface with the **Scripts** and **Tests** tabs selected. The **Scripts** tab shows an **After-response** script:

```
1 let corpo = {};
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }
3
4 - insomnia.test('Status HTTP 200', () => {
5   insomnia.expect(insomnia.response.code).toEqual(200);
6 });
7
8 - insomnia.test('Existe ao menos uma ferramenta cadastrada', () => {
9   insomnia.expect(corpo.data).toHaveLength(1);
10 });
11
12 // Grava no ambiente só se a resposta foi a esperada (200).
13 - if (insomnia.response.code === 200) {
14   insomnia.environment.set('grupo_id', string(corpo.data[0].grupo_id));
15 }
```

The **Tests** tab shows two tests that passed:

- PASS** Status HTTP 200 (After-response Test (0.2 ms))
- PASS** Existe Ao Menos Uma Ferramenta Cadastrada (After-response Test (0.2 ms))

Observações. Se o banco estiver vazio (seed não executado), esta requisição não grava o grupo e o cadastro (04) falha.

6.2 CRUD — fluxo de sucesso

Ciclo de vida normal de uma ferramenta: cadastrar, consultar de várias formas, editar e marcar a etiqueta como impressa. A ferramenta criada aqui (variável *ferramenta_id*) é reutilizada pelas pastas seguintes e removida (baixa lógica) na pasta 7.

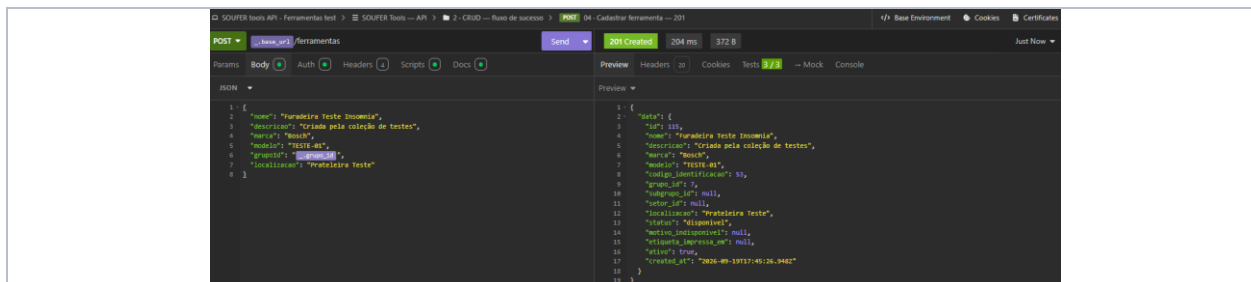
Requisição 04 · Cadastrar ferramenta (201)

O que faz. Cadastra uma ferramenta nova, chamada "Furadeira Teste Insomnia".

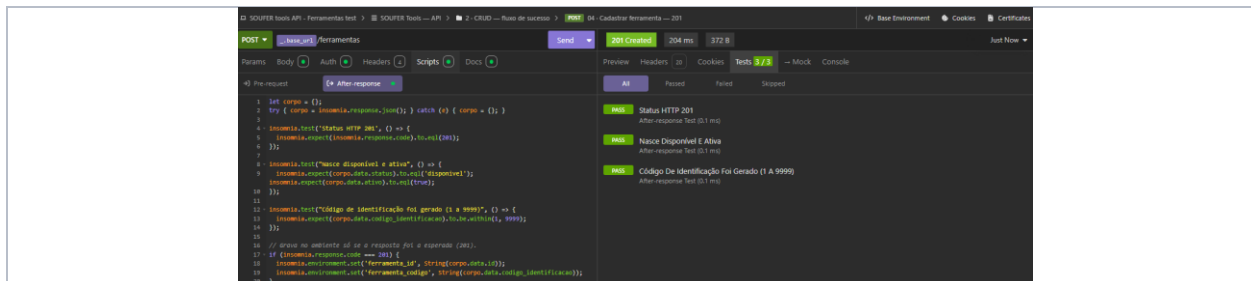
Por que existe. É o teste do "C" do CRUD (Create) e também a base dos demais: gera o *id* e o *codigo_identificacao* que as próximas requisições consultam, editam e, no fim, baixam. Assim os testes usam um registro nosso e não mexem nas ferramentas do seed.

Método e rota	<i>POST /v1/ferramentas</i>
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Corpo (JSON)	<pre>{ "nome": "Furadeira Teste Insomnia", "descricao": "Criada pela coleção de testes", "marca": "Bosch", "modelo": "TESTE-01", "grupoId": "{{ _grupo_id }}", "localizacao": "Prateleira Teste" }</pre>
Grava no ambiente	<i>ferramenta_id</i> , <i>ferramenta_codigo</i>
Resultado esperado	HTTP 201. A ferramenta volta com <i>status = disponivel</i> , <i>ativo = true</i> e <i>codigo_identificacao</i> gerado automaticamente por trigger do banco.

Print 1 · Requisição e resposta. Aba Body com o JSON do cadastro. O campo *grupoId* aparece em roxo porque usa a variável de ambiente *._grupo_id* (valor 7, obtido na requisição 03). A resposta é **201 Created** (204 ms): a ferramenta nasce com id 115, *codigo_identificacao* 53 (gerado pelo banco), *status* “disponível”, *ativo* true e *motivo_indisponivel* e *etiqueta_impressa_em* nulos.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta e grava *ferramenta_id*, *ferramenta_codigo*, e aba Tests com 3/3 testes aprovados (PASS): “Status HTTP 201”; “Nasce disponível e ativa”; “Código de identificação foi gerado (1 a 9999)”.



Observações. O código de identificação não é enviado: o banco o gera (regra de negócio do projeto). Cada execução completa da coleção cria uma ferramenta nova; ela é baixada na requisição 37.

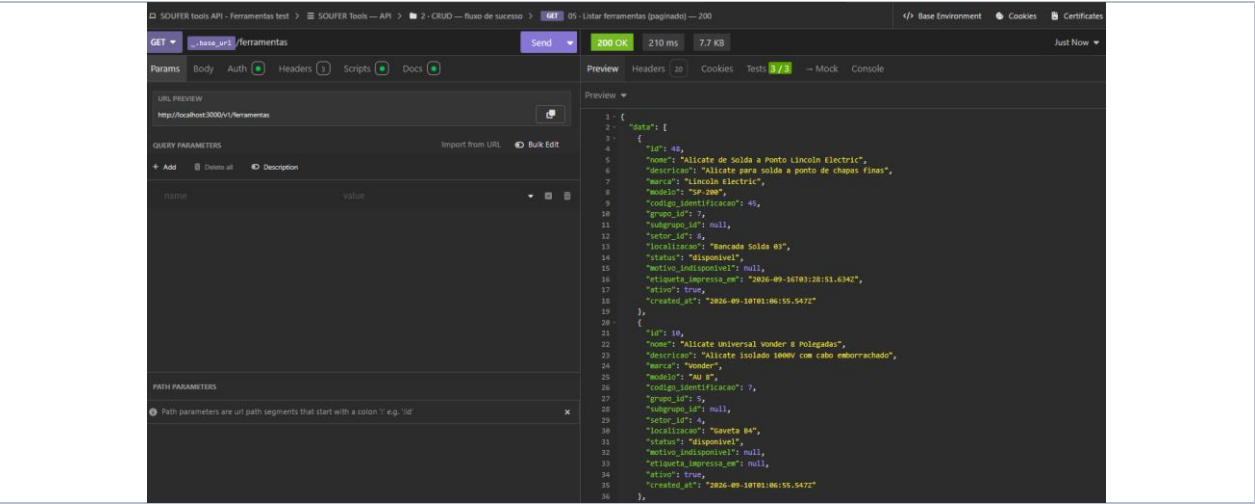
Requisição 05 · Listar ferramentas (paginado) (200)

O que faz. Lista as ferramentas ativas, com paginação padrão.

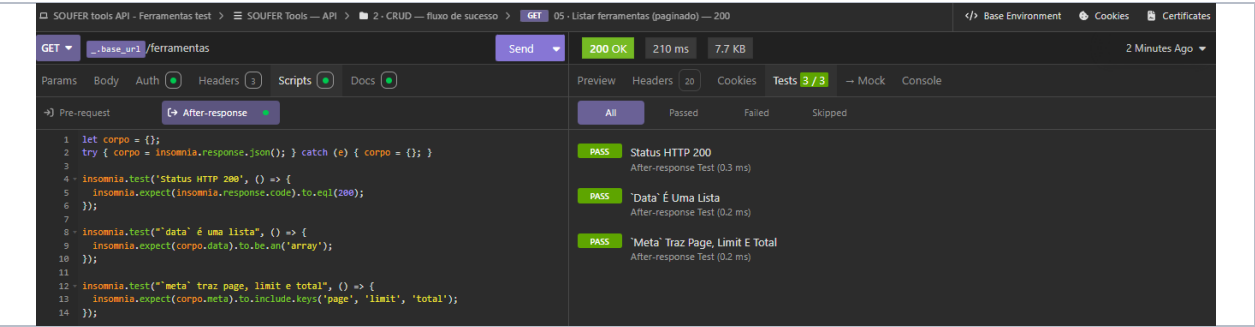
Por que existe. Teste do "R" (Read) em lista. Garante que a listagem responde com o envelope { data, meta } definido no CLAUDE.md.

Método e rota	GET /v1/ferramentas
Autenticação	Bearer Token = variável token (almoxarife)
Resultado esperado	HTTP 200 com a lista em data e meta.page, meta.Limit, meta.total.

Print 1 · Requisição e resposta. Aba Params sem nenhum parâmetro de query, o que pede a listagem padrão. A resposta 200 OK (210 ms, 7,7 KB) é a lista das ferramentas ativas; no recorte aparecem, por exemplo, o id 48 (Alicate de Solda a Ponto) e o id 10 (Alicate Universal Vonder 8 Polegadas), cada uma com todos os campos.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 3/3 testes aprovados (PASS): “Status HTTP 200”; “data é uma lista”; “meta traz page, limit e total”.



Requisição 06 · Listar com filtros, busca e ordenação (200)

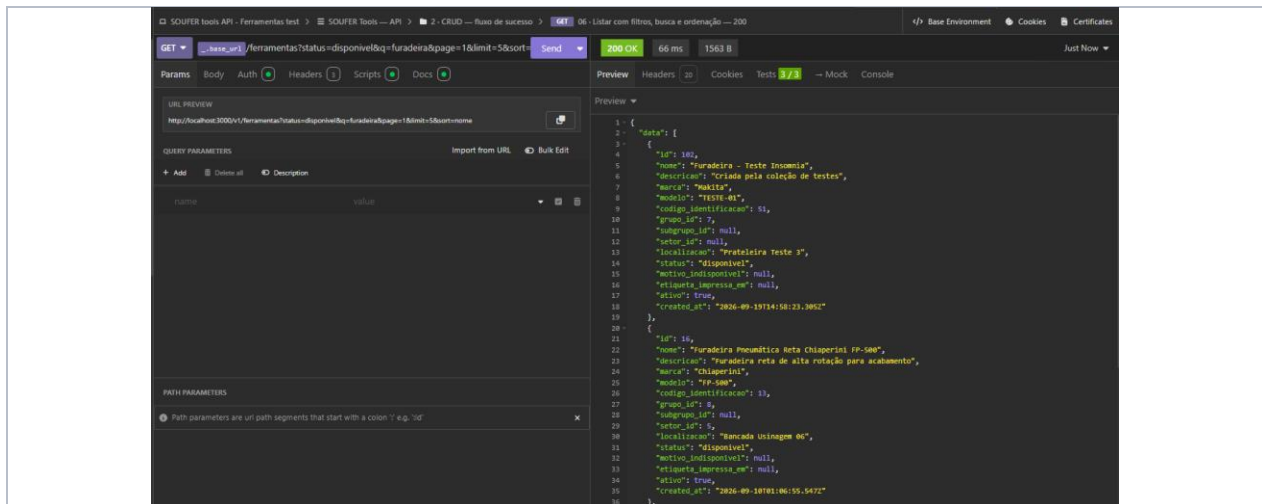
O que faz. Lista ferramentas disponíveis cujo nome, descrição, marca ou modelo contenha "furadeira", 5 por página, ordenadas por nome.

Por que existe. Cobre todos os parâmetros de query da listagem (*status*, *q*, *page*, *Limit*, *sort*). É o que as telas de busca do front vão usar.

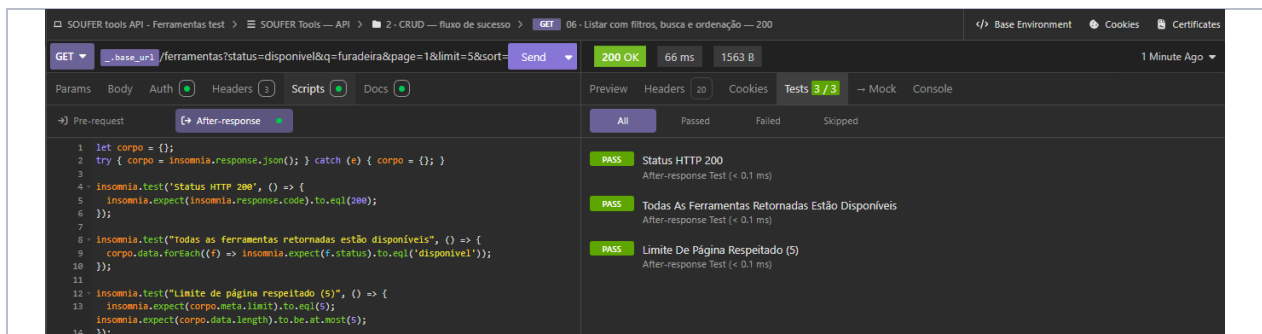
Método e rota	GET /v1/ferramentas
Query	status=disponivel&q=furadeira&page=1&limit=5&sort=nome
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 200. Só ferramentas <i>disponivel</i> , no máximo 5 itens.

Print 1 · Requisição e resposta. Aba Params com a URL pré-visualizada

.../ferramentas?status=disponivel&q=furadeira&page=1&limit=5&sort=nome. A resposta **200 OK** (66 ms, 1,5 KB) traz apenas furadeiras disponíveis, como o id 102 ("Furadeira - Teste Insomnia", registro criado em uma rodada anterior de testes) e o id 16 (Furadeira Pneumática Reta Chiaperini FP-500).



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 3/3 testes aprovados (PASS): "Status HTTP 200"; "Todas as ferramentas retornadas estão disponíveis"; "Limite de página respeitado (5)".



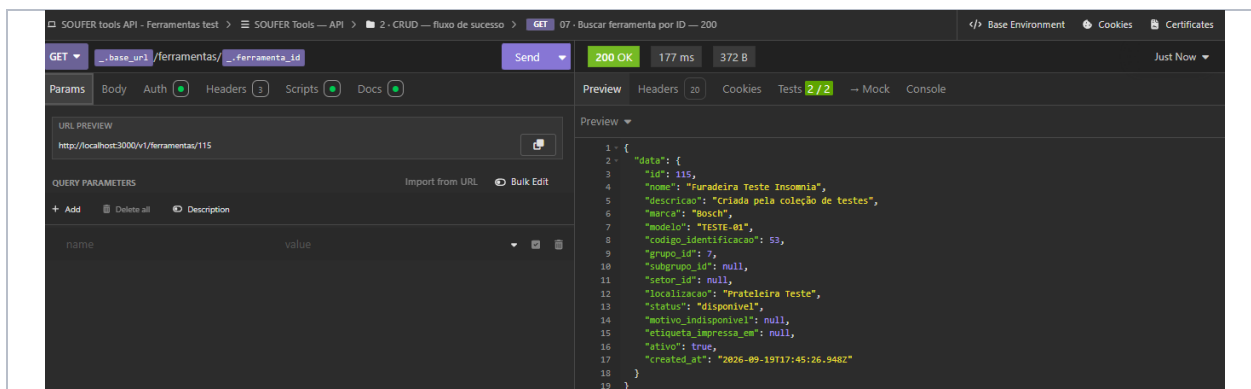
Requisição 07 · Buscar ferramenta por ID (200)

O que faz. Busca a ferramenta criada na requisição 04 pelo seu *id*.

Por que existe. Teste do "R" (Read) de um registro. Confirma que o que foi gravado no cadastro é o que a API devolve.

Método e rota	<i>GET /v1/ferramentas/:id</i>
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 200 com os dados completos da ferramenta.

Print 1 · Requisição e resposta. Aba Params. A URL usa a variável *_.ferramenta_id* e a pré-visualização mostra *.../ferramentas/115*. A resposta **200 OK** (177 ms) devolve exatamente a ferramenta criada na requisição 04: id 115, “Furadeira Teste Insomnia”, código 53, marca Bosch.



The screenshot shows the Insomnia API client interface. The URL is `http://localhost:3000/v1/ferramentas/115`. The response is a 200 OK status with a 177 ms response time and 372 B of data. The JSON response body is as follows:

```
1 {
2   "data": {
3     "id": 115,
4     "nome": "Furadeira Teste Insomnia",
5     "descricao": "Criada pela coleção de testes",
6     "marca": "Bosch",
7     "modelo": "TESTE-01",
8     "codigo_identificacao": 53,
9     "grupo_id": 7,
10    "subgrupo_id": null,
11    "setor_id": null,
12    "localizacao": "Prateleira Teste",
13    "status": "disponivel",
14    "motivo_indisponivel": null,
15    "etiqueta_impressa_em": null,
16    "ativo": true,
17    "created_at": "2026-09-19T17:45:26.948Z"
18  }
19 }
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 200”; “Retornou a ferramenta criada”.



The screenshot shows the Insomnia API client interface with the Scripts and Tests tabs selected. The Scripts tab shows a script that validates the response, and the Tests tab shows two passing tests.

Scripts (After-response):

```
1 let corpo = {};
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }
3
4 - insomnia.test('Status HTTP 200', () => {
5   insomnia.expect(insomnia.response.code).to.eq(200);
6 });
7
8 - insomnia.test('Retornou a ferramenta criada', () => {
9   insomnia.expect(String(corpo.data.id)).to.eq(insomnia.environment.get('ferramenta_id'));
10  insomnia.expect(corpo.data.nome).to.eq('Furadeira Teste Insomnia');
11 });
```

Tests:

- PASS** Status HTTP 200 (After-response Test (0.1 ms))
- PASS** Retornou A Ferramenta Criada (After-response Test (< 0.1 ms))

Observações. Depende da requisição 04 (variável *ferramenta_id*).

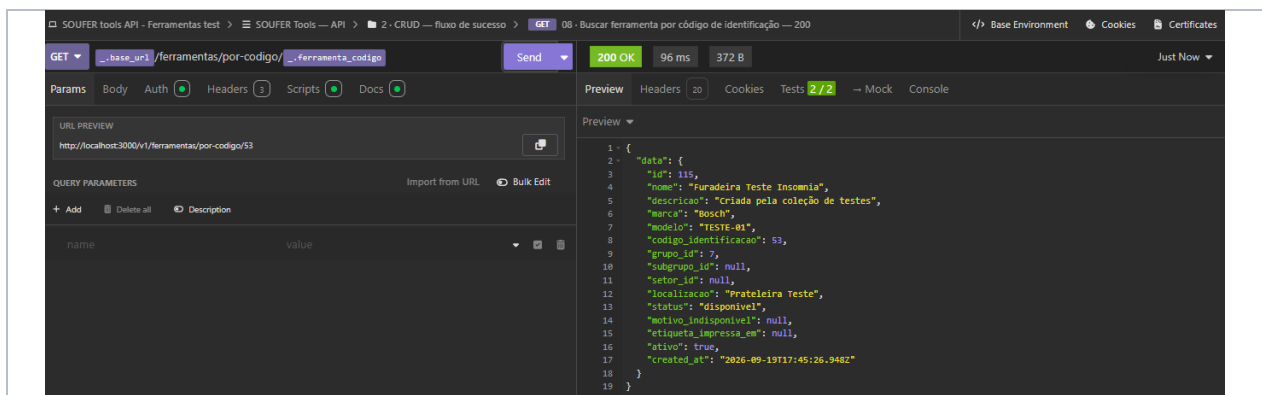
Requisição 08 · Buscar ferramenta por código de identificação (200)

O que faz. Busca a ferramenta pelo código numérico de identificação (1 a 9999).

Por que existe. É a rota usada na leitura do código na retirada/devolução no balcão (leitor de código de barras ou código gravado na ferramenta, ver CLAUDE.md). Validamos que o código gerado no cadastro localiza a mesma ferramenta.

Método e rota	<code>GET /v1/ferramentas/por-codigo/:codigo</code>
Autenticação	Bearer Token = variável <code>token</code> (almoxarife)
Resultado esperado	HTTP 200 com a ferramenta que tem esse código.

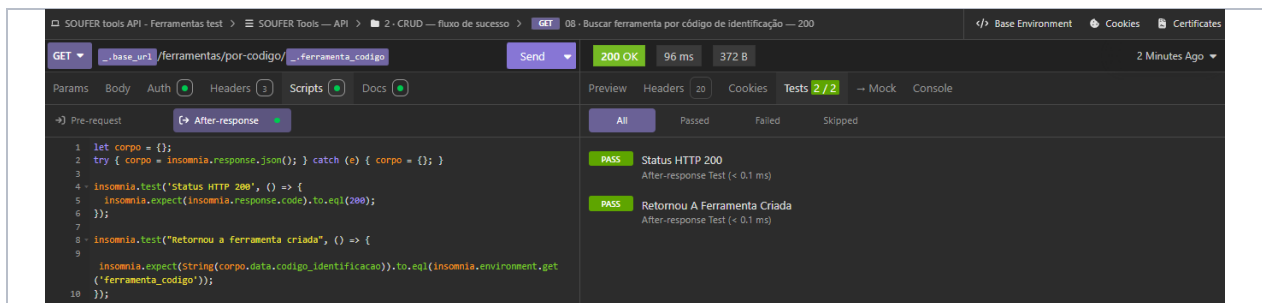
Print 1 · Requisição e resposta. A URL usa a variável `_.ferramenta_codigo` e a pré-visualização mostra `.../ferramentas/por-codigo/53`. A resposta **200 OK** (96 ms) devolve a mesma ferramenta (id 115), o que prova que o código 53 gerado pelo banco localiza a ferramenta criada.



The screenshot shows the SOUFER API client interface. The URL bar displays `http://localhost:3000/v1/ferramentas/por-codigo/53`. The response status is **200 OK** with a response time of 96 ms and a body size of 372 B. The response body is a JSON object:

```
1 {
2   "data": {
3     "id": 115,
4     "nome": "Furadeira Teste Insomnia",
5     "descricao": "Criada pela coleção de testes",
6     "marca": "Bosch",
7     "modelo": "TESTE-01",
8     "codigo_identificacao": 53,
9     "grupo_id": 7,
10    "subgrupo_id": null,
11    "setor_id": null,
12    "localizacao": "Prateleira Teste",
13    "status": "disponivel",
14    "multivo_indisponivel": null,
15    "etiqueta impressa_em": null,
16    "ativo": true,
17    "created_at": "2026-09-19T17:45:26.948Z"
18  }
19 }
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 200”; “Retornou a ferramenta criada”.



The screenshot shows the SOUFER API client interface with the 'After-response' script and test results. The script is:

```
1 let corpo = {};
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }
3
4 insomnia.test("Status HTTP 200", () => {
5   insomnia.expect(insomnia.response.code).toEqual(200);
6 });
7
8 insomnia.test("Retornou a ferramenta criada", () => {
9   insomnia.expect(String(corpo.data.codigo_identificacao)).toEqual(insomnia.environment.get(
10     'ferramenta_codigo'
11   ));
12 });
```

The test results show two tests passed:

- PASS** Status HTTP 200 (After-response Test (< 0.1 ms))
- PASS** Retornou A Ferramenta Criada (After-response Test (< 0.1 ms))

Observações. Depende da requisição 04 (variável `ferramenta_codigo`).

Requisição 09 · Consultar histórico da ferramenta (200)

O que faz. Retorna o histórico de empréstimos e ocorrências da ferramenta.

Por que existe. Cobre a rota de histórico. Como a ferramenta acabou de ser criada, esperamos um histórico vazio, o que prova que a rota funciona mesmo sem movimentações.

Método e rota	<i>GET /v1/ferramentas/:id/historico</i>
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 200 com o histórico (vazio para ferramenta nova).

Print 1 · Requisição e resposta. Requisição *GET /ferramentas/13/historico*. Neste print foi consultada a ferramenta de id 13 (Furadeira de Impacto Bosch GSB 13 RE, do seed), que já tem movimentação, para mostrar um histórico com conteúdo. A resposta **200 OK** (126 ms) traz o empréstimo 5 (colaborador Carlos Eduardo Souza, matrícula MAT001, atividade “Manutenção Preventiva”, setor Manutenção Geral, situação “atrasado”, sem data de devolução) e *ocorrencias* vazio. Para a ferramenta recém-criada o histórico viria vazio.

The screenshot shows the SOUFER tools API interface. The URL is `http://localhost:3000/v1/ferramentas/13/historico`. The response is **200 OK** with a status of 126 ms and 754 B. The JSON response is as follows:

```
{
  "data": {
    "emprestimos": [
      {
        "id": 5,
        "data_retirada": "2026-09-09T01:06:55.612Z",
        "previao_devolucao": "2026-09-12T01:06:55.612Z",
        "data_devolucao": null,
        "condicao_devolucao": null,
        "ordem_servico": null,
        "observacoes_retirada": null,
        "observacoes_devolucao": null,
        "situacao": "atrasado",
        "ferramenta_id": 13,
        "ferramenta_nome": "Furadeira de Impacto Bosch GSB 13 RE",
        "codigo_identificacao": 13,
        "eh_kit": false,
        "item_kit_id": null,
        "item_kit_nome": null,
        "colaborador_id": 7,
        "colaborador_nome": "Carlos Eduardo Souza",
        "colaborador_matricula": "MAT001",
        "setor_id": 4,
        "setor_nome": "Manutenção Geral",
        "atividade_id": 1,
        "atividade_nome": "Manutenção Preventiva",
        "atividade_observacao": null,
        "usuario_retirada_nome": "Almoxarife Principal",
        "usuario_devolucao_nome": null
      }
    ],
    "ocorrencias": []
  }
}
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 200”; “Resposta traz *data*”.

The screenshot shows the SOUFER tools API interface with the Scripts and Tests tabs. The Scripts tab contains the following script:

```
1 let corpo = {}
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }
3
4 - insomnia.test('Status HTTP 200', () => {
5   insomnia.expect(insomnia.response.code).toEqual(200);
6 });
7
8 - insomnia.test('Resposta traz `data`', () => {
9   insomnia.expect(corpo).to.have.property('data');
10 });
```

The Tests tab shows two tests, both of which passed:

- PASS Status HTTP 200** (After-response Test (0.1 ms))
- PASS Resposta Traz 'Data'** (After-response Test (< 0.1 ms))

Observações. Depende da requisição 04.

Requisição 10 · Atualizar ferramenta (PATCH, campos parciais) (200)

O que faz. Altera só a marca e a localização da ferramenta criada.

Por que existe. Teste do "U" (Update) do CRUD. Enviamos poucos campos de propósito para provar que os demais (nome, grupo etc.) permanecem intactos.

Método e rota	<i>PATCH</i> /v1/ferramentas/:id
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Corpo (JSON)	<pre>{ "marca": "Makita", "localizacao": "Prateleira Teste 2" }</pre>
Resultado esperado	HTTP 200 com a ferramenta atualizada.

Print 1 · Requisição e resposta. Aba Body com um *PATCH* que envia só *marca* e *Localizacao*. A resposta *200 OK* (240 ms) mostra a ferramenta 115 com marca “Makita” e localização “Prateleira Teste 2”, enquanto os demais campos (nome, modelo, grupo, status) continuam iguais. É a prova de que a atualização é parcial.

The screenshot shows the Insomnia client interface. The top bar indicates the request is a PATCH to the endpoint `/v1/ferramentas/:ferramenta_id`. The status is **200 OK** with a response time of 240 ms and a body size of 375 B. The request body is a JSON object:

```
{
  "marca": "Makita",
  "localizacao": "Prateleira Teste 2"
}
```

. The response body is a JSON object representing the updated tool:

```
{
  "data": {
    "id": 115,
    "nome": "Furadeira Teste Insomnia",
    "descricao": "Criada pela coleção de testes",
    "marca": "Makita",
    "modelo": "TESTE-01",
    "codigo_identificacao": 53,
    "grupo_id": 7,
    "subgrupo_id": null,
    "setor_id": null,
    "localizacao": "Prateleira Teste 2",
    "status": "disponivel",
    "ativo_indisponivel": null,
    "etiqueta_impressao_m": null,
    "ativo": true,
    "created_at": "2026-09-19T17:45:26.948Z"
  }
}
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 3/3 testes aprovados (PASS): “Status HTTP 200”; “Campos enviados foram alterados”; “Campo não enviado permaneceu igual”.

The screenshot shows the Insomnia client interface with the **After-response** tab selected. The script contains the following code:

```
1 let corpo = {};
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }
3
4 - insomnia.test('Status HTTP 200', () => {
5   insomnia.expect(insomnia.response.code).to.eql(200);
6 });
7
8 - insomnia.test('Campos enviados foram alterados', () => {
9   insomnia.expect(corpo.data.marca).to.eql('Makita');
10  insomnia.expect(corpo.data.localizacao).to.eql('Prateleira Teste 2');
11 });
12
13 - insomnia.test('Campo não enviado permaneceu igual', () => {
14   insomnia.expect(corpo.data.nome).to.eql('Furadeira Teste Insomnia');
15 });
```

. The **Tests** tab shows three tests, all of which passed:

- PASS** Status HTTP 200 (After-response Test (< 0.1 ms))
- PASS** Campos Enviados Foram Alterados (After-response Test (< 0.1 ms))
- PASS** Campo Não Enviado Permaneceu Igual (After-response Test (< 0.1 ms))

Observações. Convenção REST: atualização parcial é PATCH (PUT seria substituição completa). Esta rota era PUT e foi trocada por PATCH na API; o PUT deixou de existir e responde 404. Veja docs/api-07-put-para-patch.md.

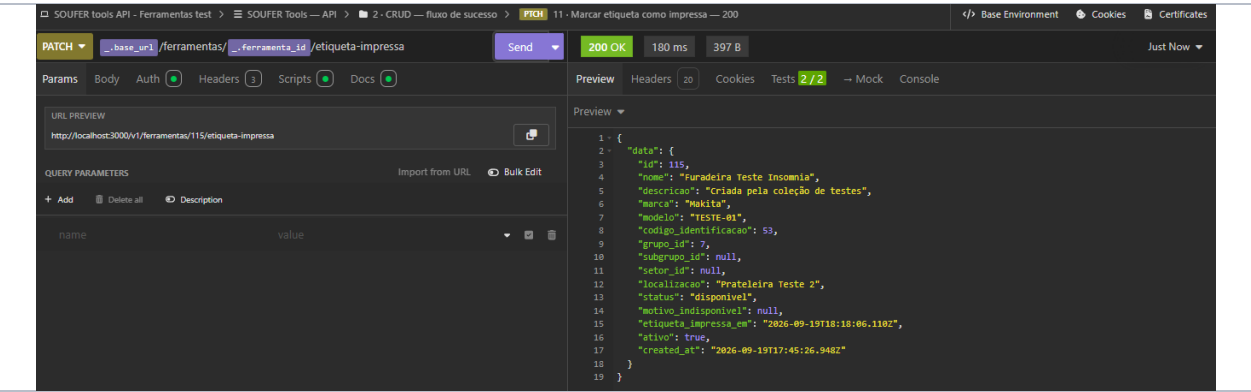
Requisição 11 · Marcar etiqueta como impressa (200)

O que faz. Registra a data e hora em que a etiqueta com o código de barras da ferramenta foi impressa.

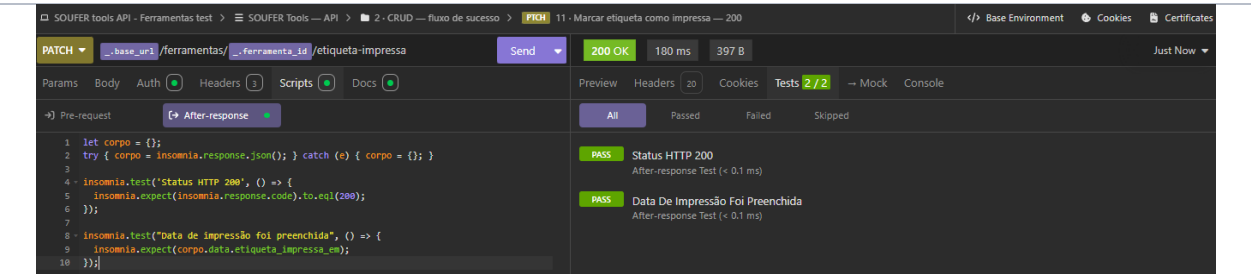
Por que existe. Permite ao almoxarife saber quais ferramentas já têm etiqueta. É uma ação específica de um único campo, por isso é *PATCH* numa sub-rota (não é um cadastro completo).

Método e rota	<i>PATCH</i> /v1/ferramentas/:id/etiqueta-impressa
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 200 com <i>etiqueta_impressa_em</i> preenchido.

Print 1 · Requisição e resposta. Requisição *PATCH* .../ferramentas/115/etiqueta-impressa, sem corpo. A resposta **200 OK** (180 ms) devolve a ferramenta com *etiqueta_impressa_em* preenchido (2026-09-19T18:18:06.110Z), antes nulo, e sem mudança no status.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 200”; “Data de impressão foi preenchida”.



Observações. Não altera o status da ferramenta.

6.3 Regras de negócio (status e conflitos 409)

Testa as regras que o projeto considera inegociáveis: mudança de status, empréstimo aberto impede a baixa e uma ferramenta só volta de "indisponível" por ação explícita. Usa dados do seed. As requisições marcadas "(auxiliar)" só localizam um id e gravam em variável; não alteram nada.

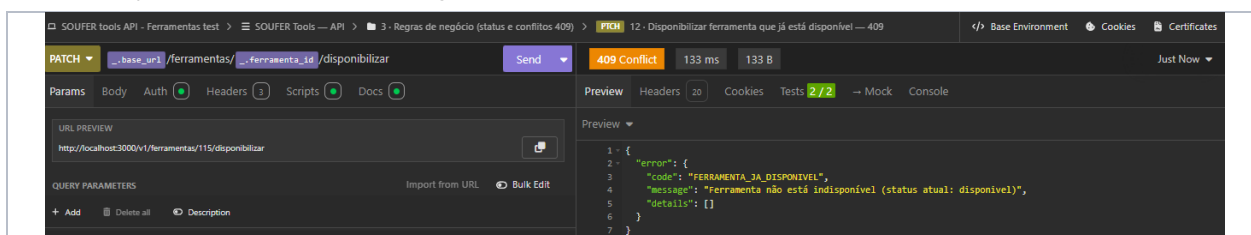
Requisição 12 · Disponibilizar ferramenta que já está disponível (409)

O que faz. Tenta "disponibilizar" uma ferramenta que já está *disponível*.

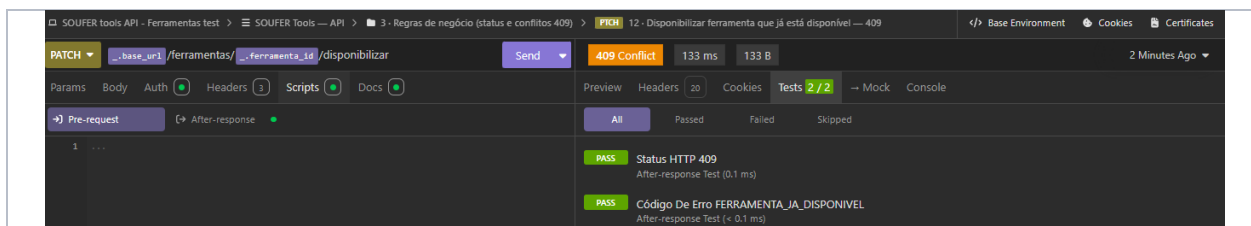
Por que existe. A rota *disponibilizar* só faz sentido para ferramentas *indisponível* (pós-reparo). Este teste garante que a API recusa a operação em outro estado, em vez de gravar auditoria sem necessidade.

Método e rota	<i>PATCH</i> /v1/ferramentas/:id/disponibilizar
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 409 (conflito de estado) com o código <i>FERRAMENTA_JA_DISPONIVEL</i> .

Print 1 · Requisição e resposta. Requisição *PATCH* .../ferramentas/115/disponibilizar sobre a ferramenta criada, que está disponível. A resposta é **409 Conflict** (133 ms) com o código *FERRAMENTA_JA_DISPONIVEL* e a mensagem “Ferramenta não está indisponível (status atual: disponível)”. A API recusa a operação porque só se disponibiliza o que está indisponível.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 409”; “Código de erro *FERRAMENTA_JA_DISPONIVEL*”. Neste print a sub-aba Pre-request aparece selecionada (vazia); o script de testes está na sub-aba After-response, marcada com o ponto verde.



Observações. Usa a ferramenta criada na requisição 04, que está *disponível*.

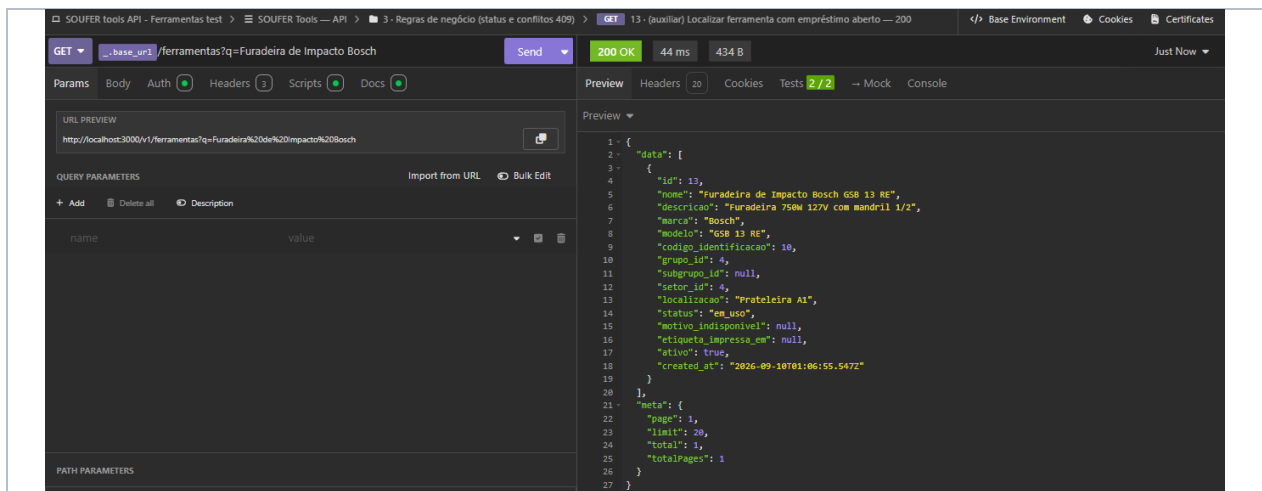
Requisição 13 · (auxiliar) Localizar ferramenta com empréstimo aberto (200)

O que faz. Procura a "Furadeira de Impacto Bosch GSB 13 RE" e grava o id dela.

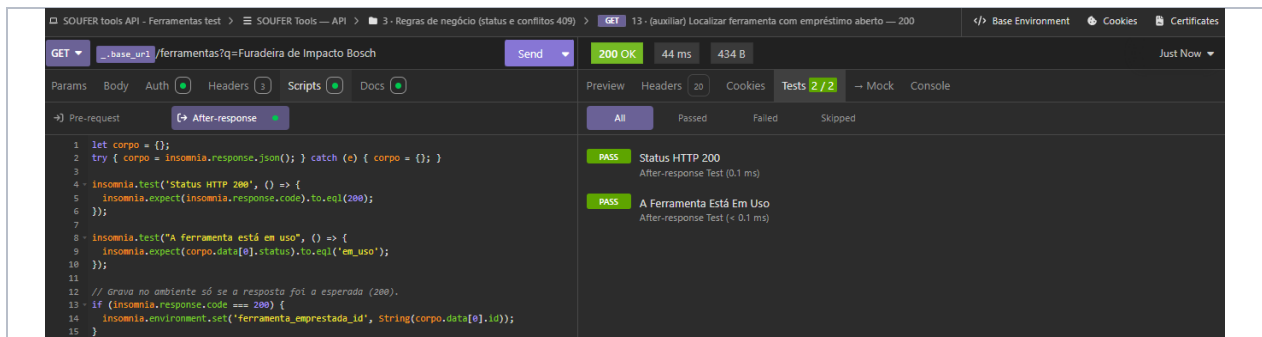
Por que existe. O seed deixa essa ferramenta emprestada (status *em_uso*). Precisamos do id dela para testar a regra de que não se pode dar baixa em ferramenta com empréstimo aberto (requisição seguinte).

Método e rota	GET /v1/ferramentas
Query	q=Furadeira de Impacto Bosch
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Grava no ambiente	<i>ferramenta_emprestada_id</i>
Resultado esperado	HTTP 200 com a furadeira Bosch, <i>status</i> = <i>em_uso</i> .

Print 1 · Requisição e resposta. Requisição auxiliar GET /ferramentas?q=Furadeira de Impacto Bosch (a pré-visualização da URL codifica os espaços como %20). A resposta 200 OK (44 ms) traz uma única ferramenta: id 13, Furadeira de Impacto Bosch GSB 13 RE, código 10, localização "Prateleira A1" e *status* "em_uso", ou seja, com empréstimo aberto. O id 13 é gravado em *ferramenta_emprestada_id*.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta e grava *ferramenta_emprestada_id*, e aba Tests com 2/2 testes aprovados (PASS): "Status HTTP 200", "A ferramenta está em uso".



Observações. Se essa ferramenta já foi devolvida, o teste "está em uso" falha e a requisição seguinte deixa de retornar 409. Rode o seed de novo ou escolha outra ferramenta emprestada.

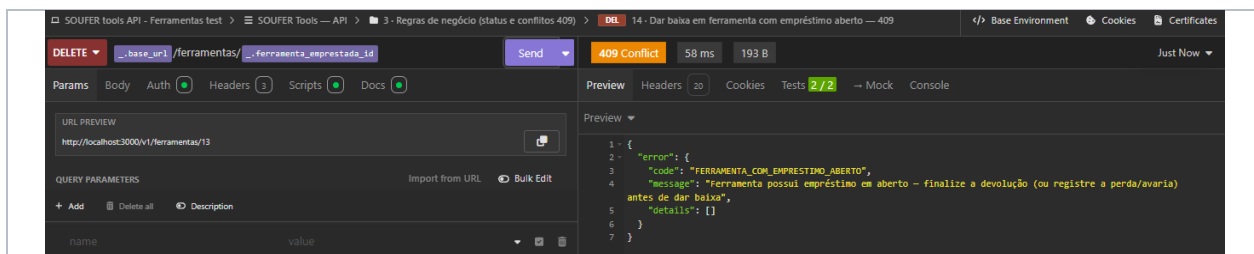
Requisição 14 · Dar baixa em ferramenta com empréstimo aberto (409)

O que faz. Tenta dar baixa (*DELETE*) na ferramenta emprestada.

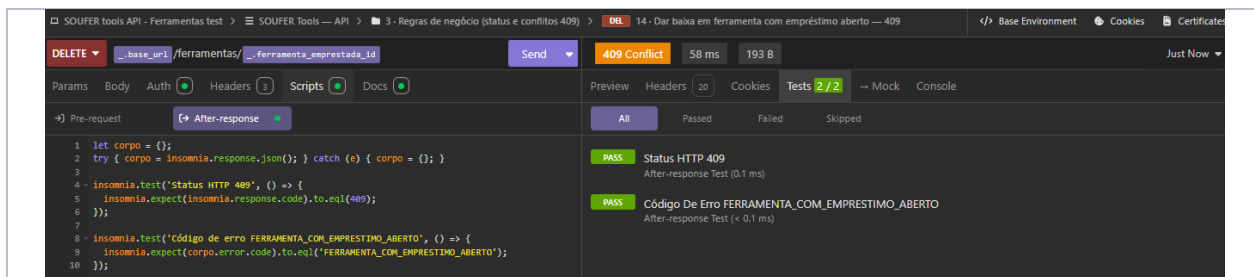
Por que existe. Regra de negócio: uma ferramenta só pode ser baixada depois que o empréstimo em aberto for finalizado, para não deixar um empréstimo sem devolução apontando para uma ferramenta inativa.

Método e rota	<i>DELETE</i> /v1/ferramentas/:id
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 409 com o código <i>FERRAMENTA_COM_EMPRESTIMO_ABERTO</i> . Nada é alterado no banco.

Print 1 · Requisição e resposta. Requisição *DELETE .../ferramentas/13*, com a URL montada pela variável *_.ferramenta_emprestada_id*. A resposta é **409 Conflict** (58 ms) com *FERRAMENTA_COM_EMPRESTIMO_ABERTO* e a mensagem que orienta a finalizar a devolução (ou registrar perda ou avaria) antes de dar baixa. Nada é alterado no banco.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 409”; “Código de erro *FERRAMENTA_COM_EMPRESTIMO_ABERTO*”.



Requisição 15 · (auxiliar) Localizar ferramenta indisponível (200)

O que faz. Busca uma ferramenta em status *indisponível* e grava o id dela.

Por que existe. A próxima requisição precisa de uma ferramenta indisponível. O seed cria algumas (por avaria ou perda numa devolução).

Método e rota	GET /v1/ferramentas
Query	status=indisponivel&limit=1
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Grava no ambiente	<i>ferramenta_indisponivel_id</i>
Resultado esperado	HTTP 200 com uma ferramenta <i>indisponivel</i> .

Print 1 · Requisição e resposta. Requisição auxiliar GET

/ferramentas?status=indisponivel&limit=1. A resposta **200 OK** (204 ms) traz a ferramenta id 24 (Furadeira de Bancada Vonder FBV 550, código 21) com *status* “indisponível” e *motivo_indisponivel* “avaria”. O *meta.total* = 4 mostra que existiam 4 ferramentas indisponíveis. O id 24 é gravado em *ferramenta_indisponivel_id*.

The screenshot shows the SOUFER tools API interface. The URL is `http://localhost:3000/v1/ferramentas?status=indisponivel&limit=1`. The query parameters are `status=indisponivel&limit=1`. The response is **200 OK** (204 ms, 458 B). The JSON body is:

```
{
  "data": [
    {
      "id": 24,
      "nome": "Furadeira de Bancada Vonder FBV 550",
      "descricao": "Furadeira de bancada 550w com mesa inclinável",
      "marca": "Vonder",
      "modelo": "FBV 550",
      "codigo_identificacao": 21,
      "grupo_id": 4,
      "subgrupo_id": null,
      "setor_id": 5,
      "localizacao": "Bancada Usinagem 01",
      "status": "indisponivel",
      "motivo_indisponivel": "avaria",
      "ferramenta_impressa_em": null,
      "ativo": true,
      "created_at": "2026-09-10T01:06:55.947Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 1,
    "total": 4,
    "totalPages": 4
  }
}
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta e grava *ferramenta_indisponivel_id*, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 200”, “Existe ao menos uma ferramenta indisponível”.

The screenshot shows the SOUFER tools API interface with the Scripts and Tests tabs. The Scripts tab shows a script that validates the response and saves the tool ID. The Tests tab shows two passing tests:

- PASS** Status HTTP 200 (After-response Test (0.2 ms))
- PASS** Existe Ao Menos Uma Ferramenta Indisponivel (After-response Test (< 0.1 ms))

Observações. A requisição seguinte libera essa ferramenta; portanto, em execuções repetidas, a lista de indisponíveis do seed vai acabando. Para criar uma nova, veja "Preparar uma ferramenta indisponível" no tutorial.

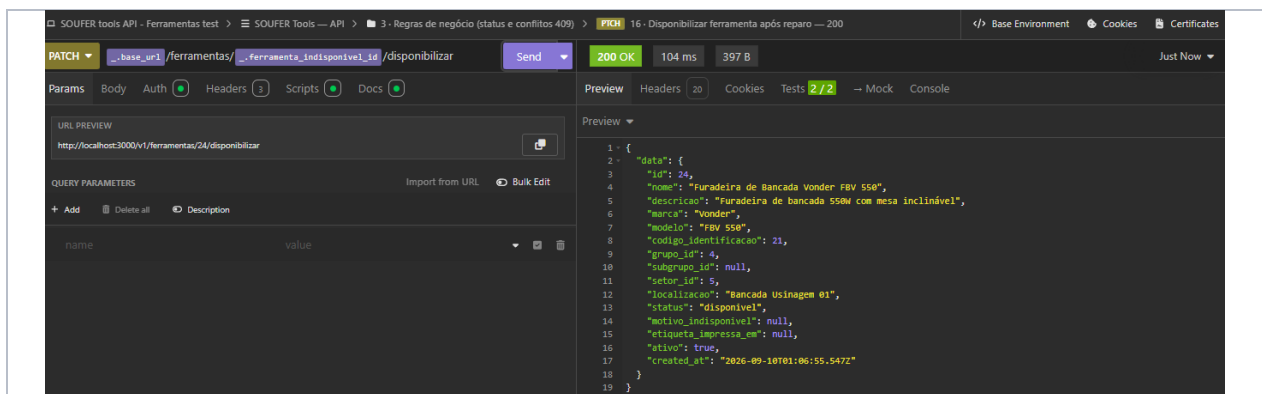
Requisição 16 · Disponibilizar ferramenta após reparo (200)

O que faz. Volta a ferramenta indisponível para *disponível*.

Por que existe. Regra de negócio: uma ferramenta só sai de *indisponível* por ação explícita e auditável do almoxarife. A API registra a ação na tabela de auditoria e resolve as ocorrências abertas dela.

Método e rota	<i>PATCH</i> <code>/v1/ferramentas/:id/disponibilizar</code>
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 200 com <i>status</i> = <i>disponível</i> e <i>motivo_indisponivel</i> = <i>null</i> .

Print 1 · Requisição e resposta. Requisição *PATCH* `.../ferramentas/24/disponibilizar` (URL montada pela variável `_.ferramenta_indisponivel_id`). A resposta **200 OK** (104 ms) mostra a ferramenta 24 com *status* “disponível” e *motivo_indisponivel* nulo. Esta requisição altera dado do seed: a ferramenta 24 deixa de estar indisponível.



The screenshot shows the Insomnia API client interface. The URL is `http://localhost:3000/v1/ferramentas/24/disponibilizar`. The method is *PATCH*. The response status is **200 OK** with a response time of 104 ms and a body size of 397 B. The response body is a JSON object:

```
1 {
2   "data": {
3     "id": 24,
4     "nome": "Furadeira de Bancada Vonder FBV 550",
5     "descricao": "Furadeira de bancada 550W com mesa inclinável",
6     "marca": "Vonder",
7     "modelo": "FBV 550",
8     "codigo_identificacao": 21,
9     "grupo_id": 4,
10    "subgrupo_id": null,
11    "setor_id": 5,
12    "localizacao": "Bancada Usinagem 01",
13    "status": "disponível",
14    "motivo_indisponivel": null,
15    "etiqueta_impressa_em": null,
16    "ativo": true,
17    "created_at": "2025-09-18T01:06:55.547Z"
18  }
19 }
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 200”; “Status virou disponível”.



The screenshot shows the Insomnia API client interface with the *Scripts* and *Tests* tabs selected. The *Scripts* tab shows an after-response script:

```
1 let corpo = {};
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }
3
4 - insomnia.test('Status HTTP 200', () => {
5   insomnia.expect(insomnia.response.code).toEqual(200);
6 });
7
8 - insomnia.test('Status virou disponível', () => {
9   insomnia.expect(corpo.data.status).toEqual('disponível');
10  insomnia.expect(corpo.data.motivo_indisponivel);
11 });
```

The *Tests* tab shows two tests that passed:

- PASS** Status HTTP 200 (After-response Test (< 0.1 ms))
- PASS** Status virou disponível (After-response Test (< 0.1 ms))

Observações. ATENÇÃO: altera dados do seed (a ferramenta passa a disponível e suas ocorrências são resolvidas). Só rode em ambiente de desenvolvimento.

6.4 Autenticação e autorização (401 e 403)

Provam que a rota é protegida: sem token, token inválido e perfil sem permissão. Não alteram o banco.

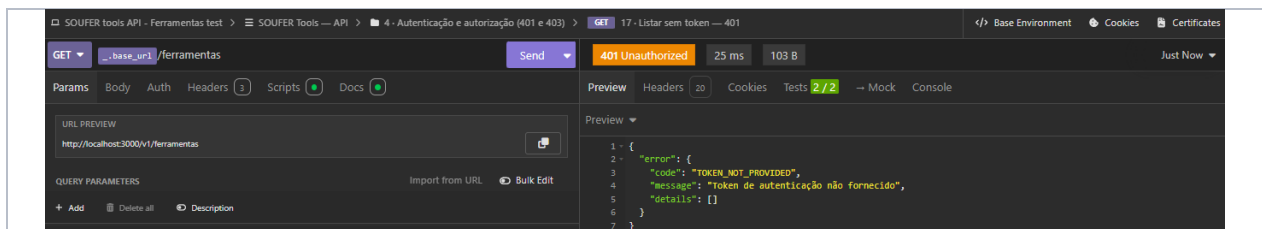
Requisição 17 · Listar sem token (401)

O que faz. Chama a listagem sem enviar o cabeçalho *Authorization*.

Por que existe. Garante que a API não expõe dados de ferramentas a quem não está autenticado.

Método e rota	GET /v1/ferramentas
Autenticação	Nenhuma (sem cabeçalho Authorization)
Resultado esperado	HTTP 401 com o código <i>TOKEN_NOT_PROVIDED</i> .

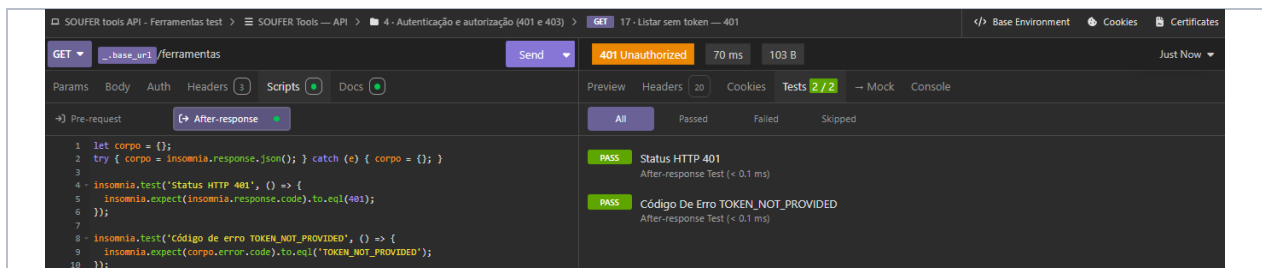
Print 1 · Requisição e resposta. Requisição *GET /ferramentas* sem autenticação: a aba Auth está sem o ponto verde. A resposta é **401 Unauthorized** (25 ms) com *TOKEN_NOT_PROVIDED* e a mensagem “Token de autenticação não fornecido”.



The screenshot shows the Insomnia client interface. The top bar indicates the request is a GET to `/v1/ferramentas` with a status of **401 Unauthorized**, a response time of 25 ms, and a size of 103 B. The 'Auth' tab is selected, showing no authentication is configured. The 'Preview' tab on the right displays the JSON response:

```
{  "error": {    "code": "TOKEN_NOT_PROVIDED",    "message": "Token de autenticação não fornecido",    "details": []  }}
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 401”; “Código de erro *TOKEN_NOT_PROVIDED*”.



The screenshot shows the Insomnia client interface with the 'Scripts' and 'Tests' tabs active. The 'Scripts' tab shows an 'After-response' script:

```
1 let corpo = {};  
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }  
3  
4 - insomnia.test('Status HTTP 401', () => {  
5   insomnia.expect(insomnia.response.code).toEqual(401);  
6 });  
7  
8 - insomnia.test('Código de erro TOKEN_NOT_PROVIDED', () => {  
9   insomnia.expect(corpo.error.code).toEqual('TOKEN_NOT_PROVIDED');  
10 });
```


The 'Tests' tab shows two tests, both passing:

- PASS** Status HTTP 401 (After-response Test < 0.1 ms)
- PASS** Código De Erro TOKEN_NOT_PROVIDED (After-response Test < 0.1 ms)

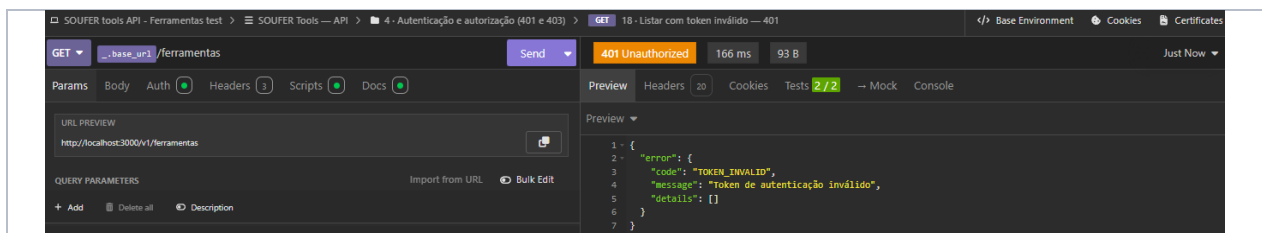
Requisição 18 · Listar com token inválido (401)

O que faz. Chama a listagem com um token que não é um JWT válido.

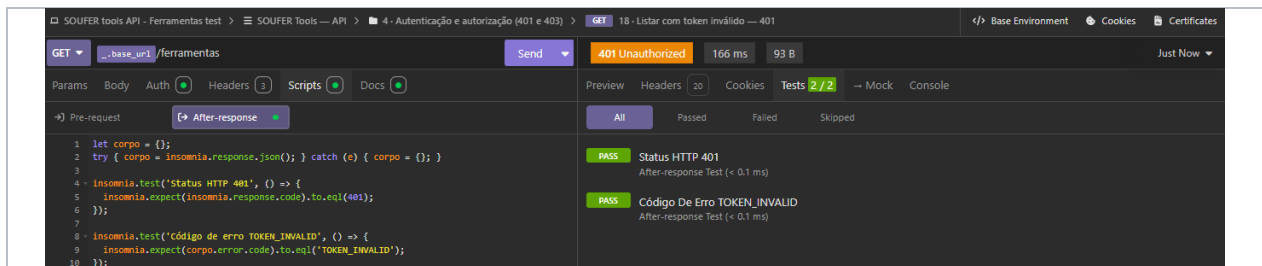
Por que existe. Garante que a API valida a assinatura do token e não aceita qualquer texto.

Método e rota	GET /v1/ferramentas
Autenticação	Bearer Token fixo inválido (<i>token.invalido.123</i>)
Resultado esperado	HTTP 401 com o código <i>TOKEN_INVALID</i> .

Print 1 · Requisição e resposta. Mesma rota, agora com a aba Auth preenchida (ponto verde) por um Bearer Token fixo e inválido. A resposta é **401 Unauthorized** (166 ms) com *TOKEN_INVALID* e a mensagem “Token de autenticação inválido”, o que prova que a API confere a assinatura do token.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 401”; “Código de erro *TOKEN_INVALID*”.



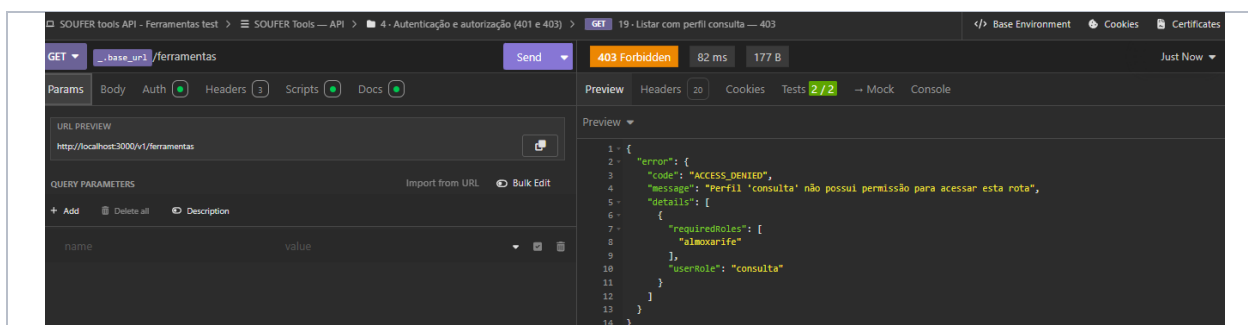
Requisição 19 · Listar com perfil consulta (403)

O que faz. Chama a listagem usando o token do perfil *consulta*.

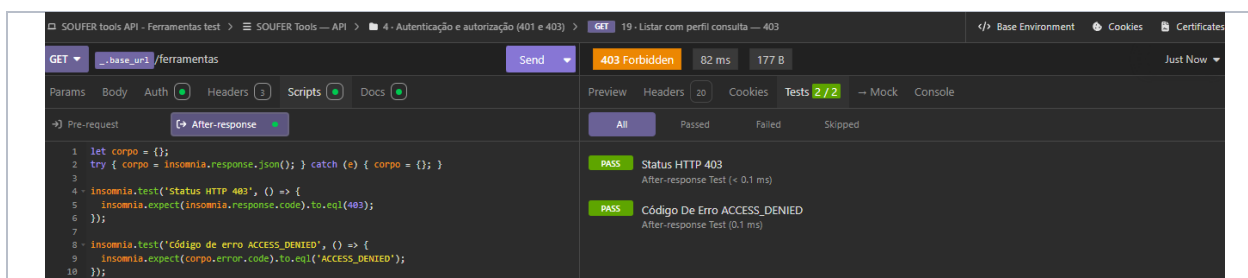
Por que existe. Regra de negócio: só o almoxarife acessa o CRUD de ferramentas; o perfil consulta é somente leitura de disponibilidade. Distingue 401 (não autenticado) de 403 (autenticado, mas sem permissão).

Método e rota	GET /v1/ferramentas
Autenticação	Bearer Token = variável <i>token_consulta</i> (perfil consulta)
Resultado esperado	HTTP 403 com o código <i>ACCESS_DENIED</i> .

Print 1 · Requisição e resposta. Mesma rota, com o token do perfil consulta. A resposta é **403 Forbidden** (82 ms) com *ACCESS_DENIED*. Em *details*, a API informa o perfil exigido (*requiredRoles*: almoxarife) e o perfil de quem chamou (*userRole*: consulta). A diferença para o 401 é que aqui o usuário está autenticado, mas não tem permissão.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 403”; “Código de erro *ACCESS_DENIED*”.



Observações. Depende da requisição 02. O token de consulta vale 15 minutos.

6.5 Validação de entrada (400)

Enviam dados inválidos de propósito para provar que a API rejeita com HTTP 400 e o envelope de erro padrão (*VALIDATION_ERROR*). Nenhuma altera o banco.

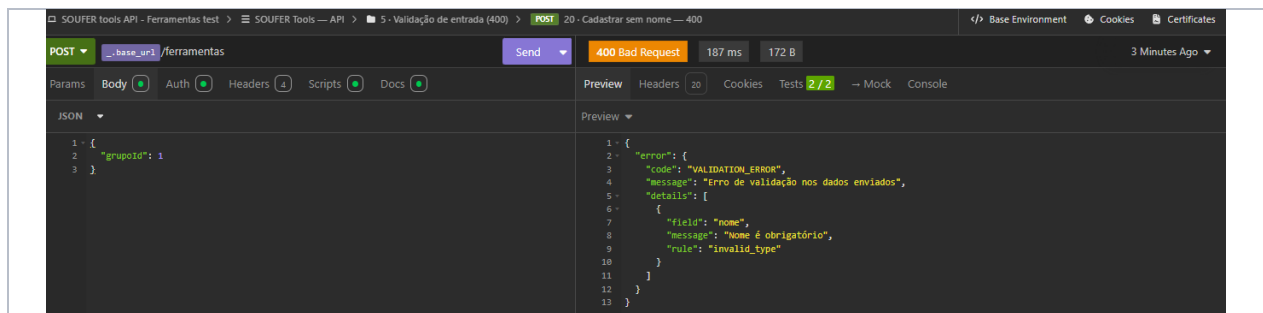
Requisição 20 · Cadastrar sem nome (400)

O que faz. Tenta cadastrar uma ferramenta sem informar o nome.

Por que existe. O nome é obrigatório. A validação (Zod) roda antes do banco, então o resultado não depende do *groupId* existir.

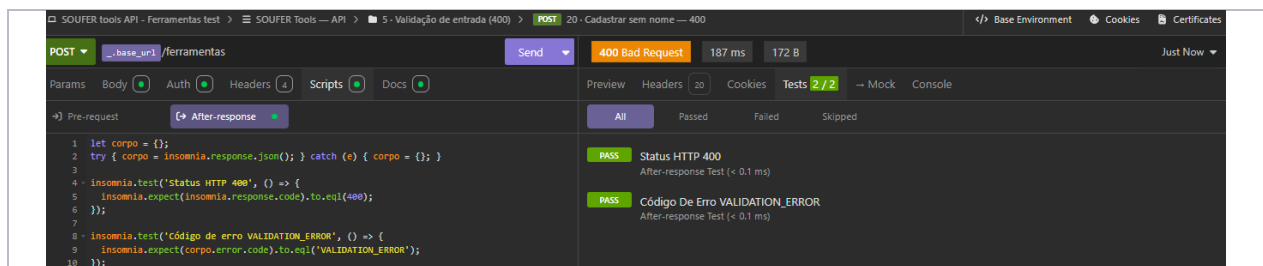
Método e rota	POST /v1/ferramentas
Autenticação	Bearer Token = variável <i>token</i> (almojarife)
Corpo (JSON)	<pre>{ "groupId": 1 }</pre>
Resultado esperado	HTTP 400 <i>VALIDATION_ERROR</i> com o campo <i>nome</i> em <i>details</i> .

Print 1 · Requisição e resposta. Aba Body com apenas `{ "groupId": 1 }`, sem o nome. A resposta é **400 Bad Request** (187 ms) com *VALIDATION_ERROR*; em *details*, o campo *nome* com a mensagem “Nome é obrigatório” (regra *invalid_type*).



The screenshot shows the SOUFER Tools API interface. The request is a POST to `/v1/ferramentas` with a JSON body `{ "groupId": 1 }`. The response is a 400 Bad Request (187 ms, 172 B). The response body is a JSON object with an error message: `{ "error": { "code": "VALIDATION_ERROR", "message": "Erro de validação nos dados enviados", "details": { "field": "nome", "message": "Nome é obrigatório", "rule": "invalid_type" } } }`.

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *VALIDATION_ERROR*”.



The screenshot shows the SOUFER Tools API interface. The request is a POST to `/v1/ferramentas` with a JSON body `{ "groupId": 1 }`. The response is a 400 Bad Request (187 ms, 172 B). The response body is a JSON object with an error message: `{ "error": { "code": "VALIDATION_ERROR", "message": "Erro de validação nos dados enviados", "details": { "field": "nome", "message": "Nome é obrigatório", "rule": "invalid_type" } } }`. The After-response script is:

```
1 let corpo = {};  
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }  
3  
4 insomnia.test('Status HTTP 400', () => {  
5   insomnia.expect(insomnia.response.code).toEqual(400);  
6 });  
7  
8 insomnia.test('Código de erro VALIDATION_ERROR', () => {  
9   insomnia.expect(corpo.error.code).toEqual("VALIDATION_ERROR");  
10 });
```

 The Tests section shows 2/2 tests passed (PASS): `Status HTTP 400` and `Código De Erro VALIDATION_ERROR`.

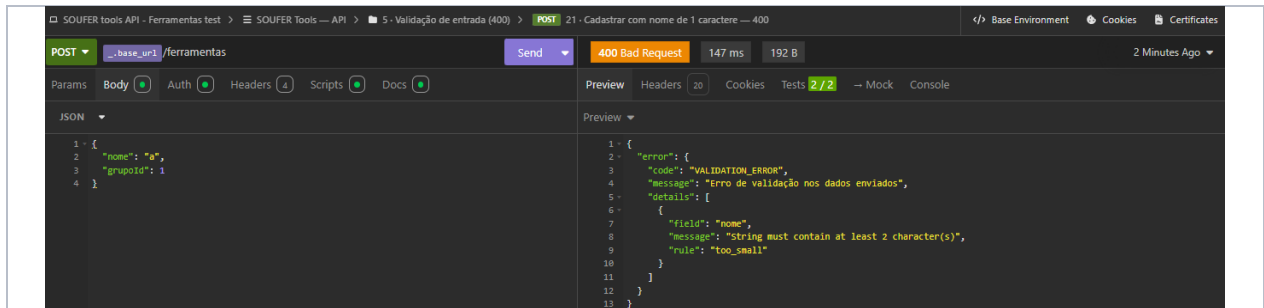
Requisição 21 · Cadastrar com nome de 1 caractere (400)

O que faz. Tenta cadastrar com um nome curto demais.

Por que existe. Cobre o limite mínimo de tamanho do nome (2 a 150 caracteres).

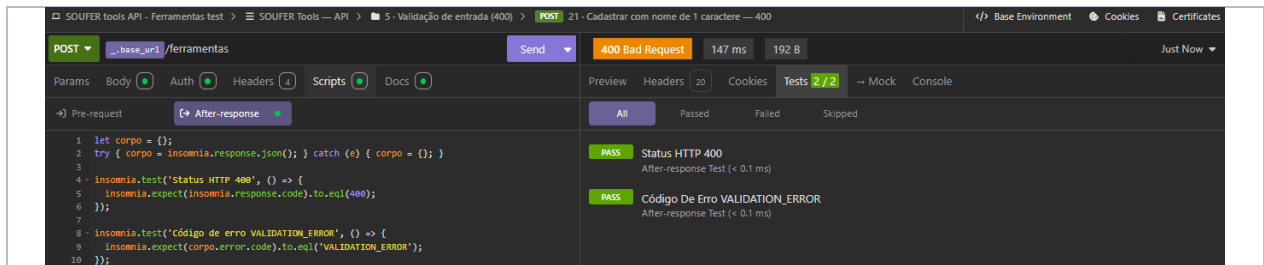
Método e rota	<i>POST</i> /v1/ferramentas
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Corpo (JSON)	<pre>{ "nome": "a", "grupoId": 1 }</pre>
Resultado esperado	HTTP 400 <i>VALIDATION_ERROR</i> .

Print 1 · Requisição e resposta. Aba Body com `{ "nome": "a", "grupoId": 1 }`. A resposta é **400 Bad Request** (147 ms) com *VALIDATION_ERROR*; o campo *nome* recebe a mensagem “String must contain at least 2 character(s)” (regra *too_small*), porque o nome exige no mínimo 2 caracteres.



The screenshot shows the Insomnia client interface. The top bar indicates the request is a POST to `/v1/ferramentas` with a status of **400 Bad Request**, a response time of 147 ms, and a body size of 192 B. The left sidebar shows the request details, including the body tab which contains the JSON payload: `{ "nome": "a", "grupoId": 1 }`. The right pane shows the response preview, which is a JSON object with an error message: `{ "error": { "code": "VALIDATION_ERROR", "message": "Erro de validação nos dados enviados", "details": { "field": "nome", "message": "String must contain at least 2 character(s)", "rule": "too_small" } } }`.

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *VALIDATION_ERROR*”.



The screenshot shows the Insomnia client interface with the **Scripts** and **Tests** tabs selected. The **Scripts** tab shows an after-response script:

```
1 let corpo = {};  
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }  
3  
4 - insomnia.test("Status HTTP 400", () => {  
5   insomnia.expect(insomnia.response.code).toEqual(400);  
6 });  
7  
8 - insomnia.test("Código de erro VALIDATION_ERROR", () => {  
9   insomnia.expect(corpo.error.code).toEqual("VALIDATION_ERROR");  
10 });
```

The **Tests** tab shows two tests that have passed: **Status HTTP 400** and **Código De Erro VALIDATION_ERROR**, both with a status of **PASS** and a duration of **After-response Test (< 0.1 ms)**.

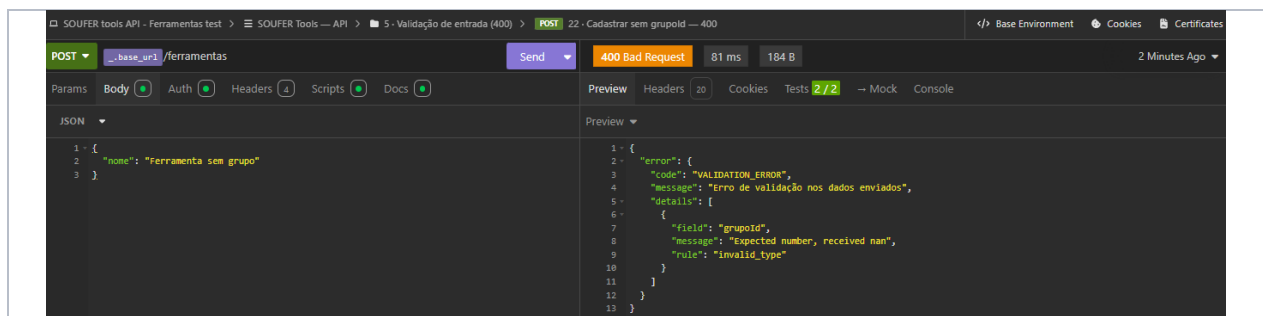
Requisição 22 · Cadastrar sem grupoid (400)

O que faz. Tenta cadastrar sem informar o grupo da ferramenta.

Por que existe. O *grupoId* é obrigatório: toda ferramenta pertence a um grupo.

Método e rota	POST /v1/ferramentas
Autenticação	Bearer Token = variável <i>token</i> (almojarife)
Corpo (JSON)	<pre>{ "nome": "Ferramenta sem grupo" }</pre>
Resultado esperado	HTTP 400 <i>VALIDATION_ERROR</i> .

Print 1 · Requisição e resposta. Aba Body com apenas o nome, sem *grupoId*. A resposta é **400 Bad Request** (81 ms) com *VALIDATION_ERROR*; o campo *grupoId* recebe “Expected number, received nan” (regra *invalid_type*): sem o valor, a conversão numérica resulta em *NaN* e é rejeitada.



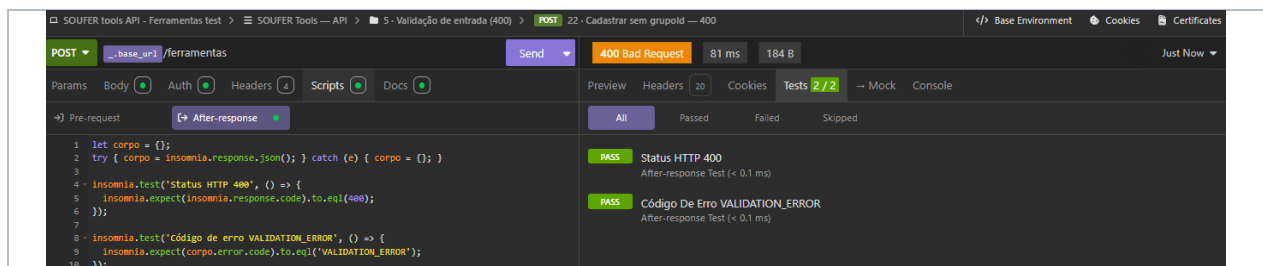
The screenshot shows the Insomnia API client interface. The top bar indicates the request is a POST to `/v1/ferramentas` with a status of **400 Bad Request**, a response time of 81 ms, and a body size of 184 B. The 'Body' tab is selected, showing the JSON request body:

```
{
  "nome": "Ferramenta sem grupo"
}
```

. The 'Preview' tab on the right shows the JSON response body:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Erro de validação nos dados enviados",
    "details": [
      {
        "field": "grupoId",
        "message": "Expected number, received nan",
        "rule": "invalid_type"
      }
    ]
  }
}
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *VALIDATION_ERROR*”.



The screenshot shows the Insomnia API client interface with the 'Scripts' and 'Tests' tabs selected. The 'Scripts' tab shows an after-response script:

```
1 let corpo = {}
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }
3
4 - insomnia.test('Status HTTP 400', () => {
5   insomnia.expect(insomnia.response.code).to.eql(400);
6 });
7
8 - insomnia.test('Código de erro VALIDATION_ERROR', () => {
9   insomnia.expect(corpo.error.code).to.eql("VALIDATION_ERROR");
10 });
```

. The 'Tests' tab shows two tests: 'Status HTTP 400' and 'Código De Erro VALIDATION_ERROR', both of which passed. The 'Preview' tab on the right shows the JSON response body:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Erro de validação nos dados enviados",
    "details": [
      {
        "field": "grupoId",
        "message": "Expected number, received nan",
        "rule": "invalid_type"
      }
    ]
  }
}
```

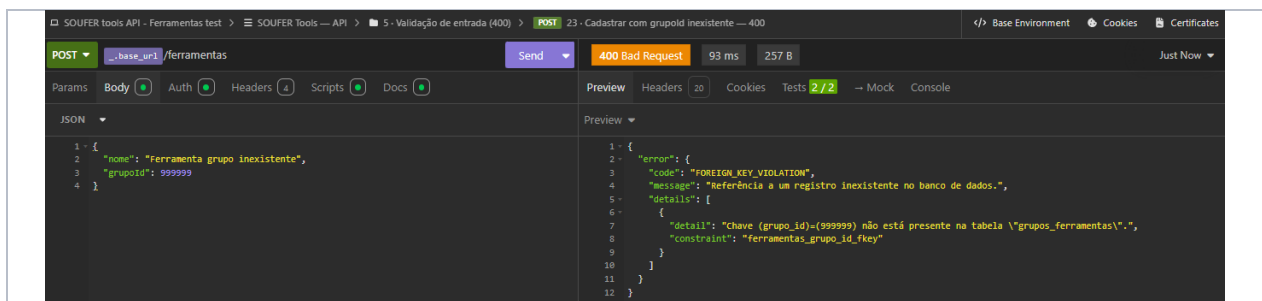
Requisição 23 · Cadastrar com grupold inexistente (400)

O que faz. Tenta cadastrar com um grupo que não existe no banco.

Por que existe. O formato do dado é válido, mas a referência não existe. Prova que a integridade referencial do banco (chave estrangeira) é respeitada e que o erro é devolvido no envelope padrão, em vez de um erro 500.

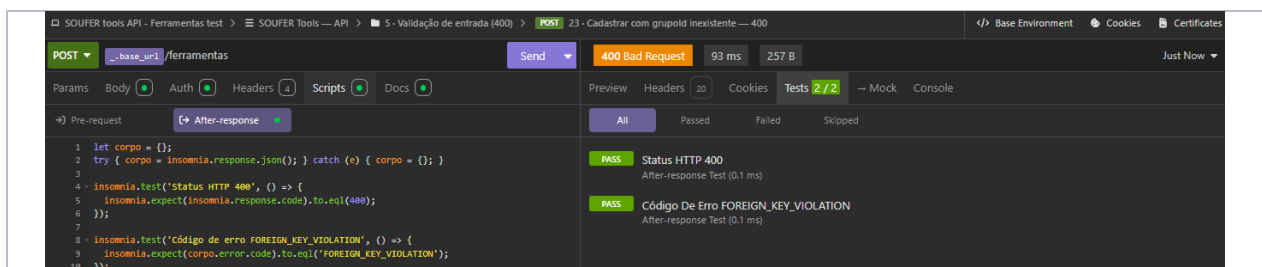
Método e rota	POST /v1/ferramentas
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Corpo (JSON)	<pre>{ "nome": "Ferramenta grupo inexistente", "grupoId": 999999 }</pre>
Resultado esperado	HTTP 400 com o código <i>FOREIGN_KEY_VIOLATION</i> . Nada é gravado.

Print 1 · Requisição e resposta. Aba Body com um *grupoId* bem formado, mas inexistente (999999). A resposta é **400 Bad Request** (93 ms) com *FOREIGN_KEY_VIOLATION* e a mensagem “Referência a um registro inexistente no banco de dados”. Em *details*, o banco informa que a chave *grupo_id* = 999999 não existe em *grupos_ferramentas* (restrição *ferramentas_grupo_id_fkey*). O dado é válido no formato, mas a integridade referencial o barra.



The screenshot shows the Insomnia interface for a POST request to `/v1/ferramentas`. The request body is a JSON object: `{ "nome": "Ferramenta grupo inexistente", "grupoId": 999999 }`. The response is a `400 Bad Request` with a status of 400, a response time of 93 ms, and a body size of 257 B. The response body is a JSON object: `{ "error": { "code": "FOREIGN_KEY_VIOLATION", "message": "Referência a um registro inexistente no banco de dados.", "details": { "detail": "Chave (grupo_id)=(999999) não está presente na tabela \"grupos_ferramentas\".", "constraint": "ferramentas_grupo_id_fkey" } } } }`.

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *FOREIGN_KEY_VIOLATION*”.



The screenshot shows the Insomnia interface for the same POST request. The **Scripts** tab is active, showing a script that validates the response status and error code. The **Tests** tab is also active, showing two tests: `Status HTTP 400` and `Código De Erro FOREIGN_KEY_VIOLATION`, both of which passed.

Observações. Pela convenção REST, um recurso referenciado inexistente costuma ser 422 ou 409. A API atual responde 400, e a coleção segue o comportamento real. Ver “Convenções REST” no tutorial.

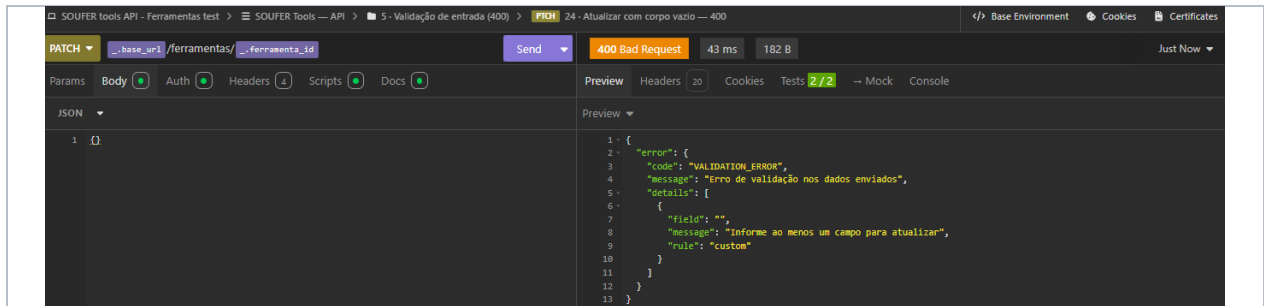
Requisição 24 · Atualizar com corpo vazio (400)

O que faz. Tenta atualizar a ferramenta sem informar nenhum campo.

Por que existe. Regra da API: uma atualização precisa alterar pelo menos um campo ("Informe ao menos um campo para atualizar").

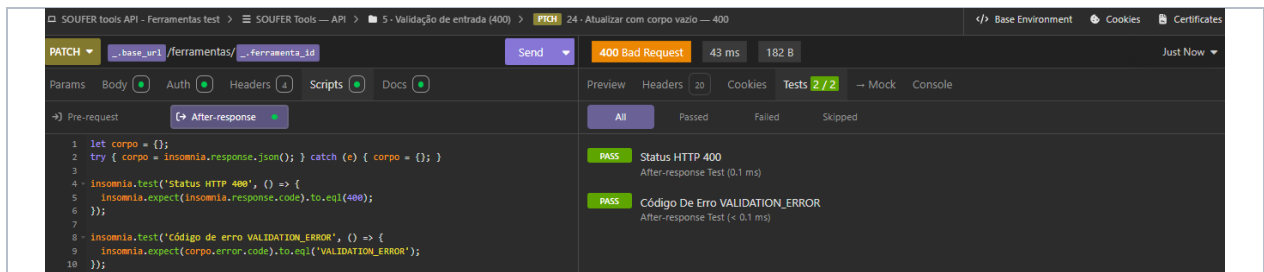
Método e rota	<code>PATCH /v1/ferramentas/:id</code>
Autenticação	Bearer Token = variável <code>token</code> (almoxarife)
Corpo (JSON)	<code>{}</code>
Resultado esperado	HTTP 400 <code>VALIDATION_ERROR</code> .

Print 1 · Requisição e resposta. Requisição `PATCH .../ferramentas/{ _ferramenta_id }` com o corpo vazio `{}`. A resposta é **400 Bad Request** (43 ms) com `VALIDATION_ERROR`; o detalhe tem `field` vazio (o erro é do objeto inteiro), a mensagem "Informe ao menos um campo para atualizar" e a regra `custom`.



The screenshot shows the SOUFER tools API interface. The request is a `PATCH` to `/v1/ferramentas/_ferramenta_id` with an empty body. The response is a `400 Bad Request` (43 ms, 182 B). The response body is a JSON object with an error message: `{ "error": { "code": "VALIDATION_ERROR", "message": "Erro de validação nos dados enviados", "details": { "field": "", "message": "Informe ao menos um campo para atualizar", "rule": "custom" } } }`.

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): "Status HTTP 400"; "Código de erro `VALIDATION_ERROR`".



The screenshot shows the SOUFER tools API interface with the Scripts and Tests tabs. The Scripts tab shows a script that validates the response. The Tests tab shows two tests: "Status HTTP 400" and "Código De Erro `VALIDATION_ERROR`", both of which passed.

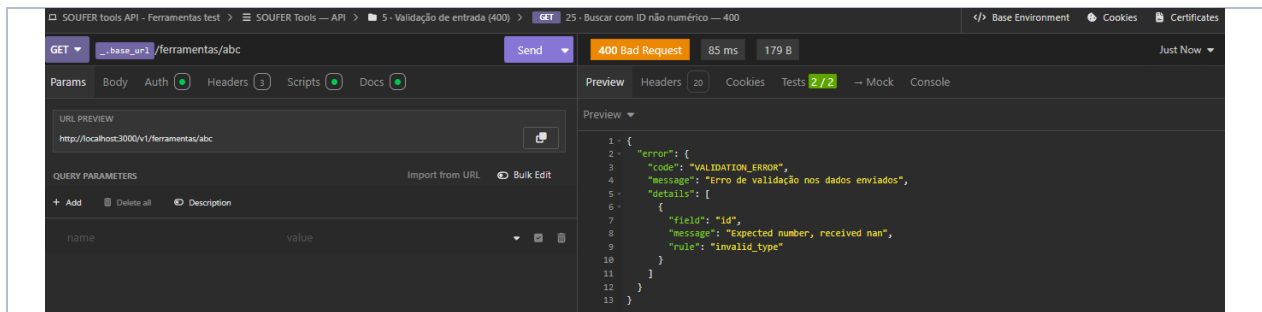
Requisição 25 · Buscar com ID não numérico (400)

O que faz. Usa o texto *abc* no lugar do id.

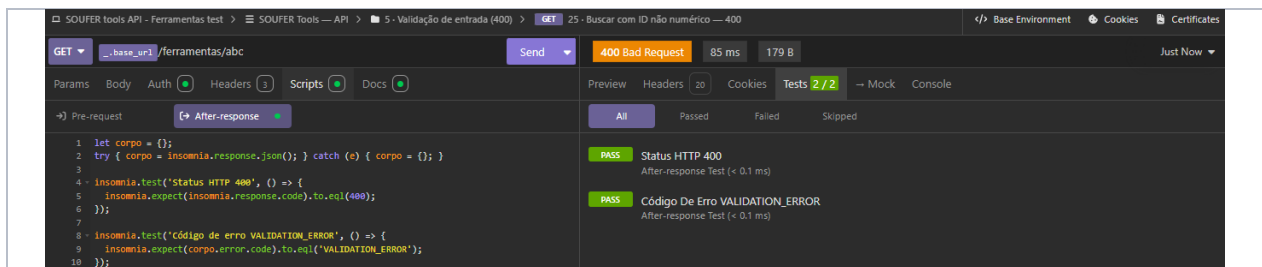
Por que existe. O id precisa ser um inteiro positivo. Garante que texto vira erro 400, e não um erro interno.

Método e rota	GET /v1/ferramentas/abc
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 400 <i>VALIDATION_ERROR</i> .

Print 1 · Requisição e resposta. Requisição *GET /ferramentas/abc*, com texto no lugar do id. A resposta é *400 Bad Request* (85 ms) com *VALIDATION_ERROR*; o campo *id* recebe “Expected number, received nan”. Texto no lugar do id vira erro 400 e não um erro interno.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *VALIDATION_ERROR*”.



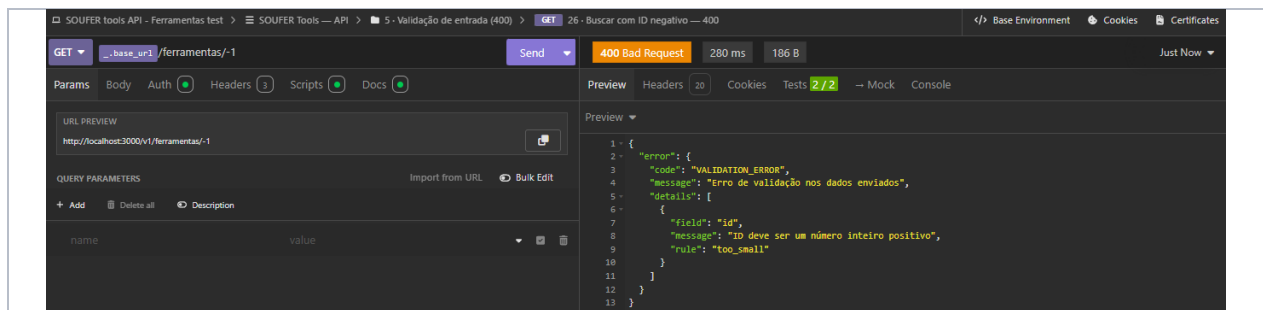
Requisição 26 · Buscar com ID negativo (400)

O que faz. Usa -1 como id.

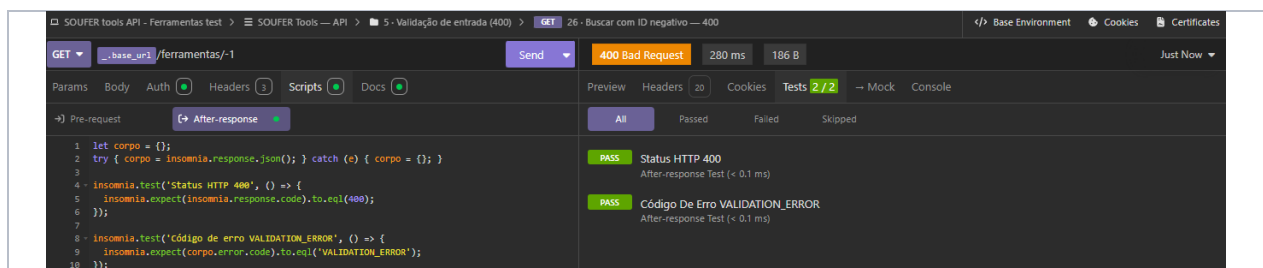
Por que existe. Cobre o limite inferior: o id deve ser positivo.

Método e rota	GET /v1/ferramentas/-1
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 400 <i>VALIDATION_ERROR</i> .

Print 1 · Requisição e resposta. Requisição *GET /ferramentas/-1*. A resposta é *400 Bad Request* (280 ms) com *VALIDATION_ERROR*; o campo *id* recebe “ID deve ser um número inteiro positivo” (regra *too_small*).



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *VALIDATION_ERROR*”.



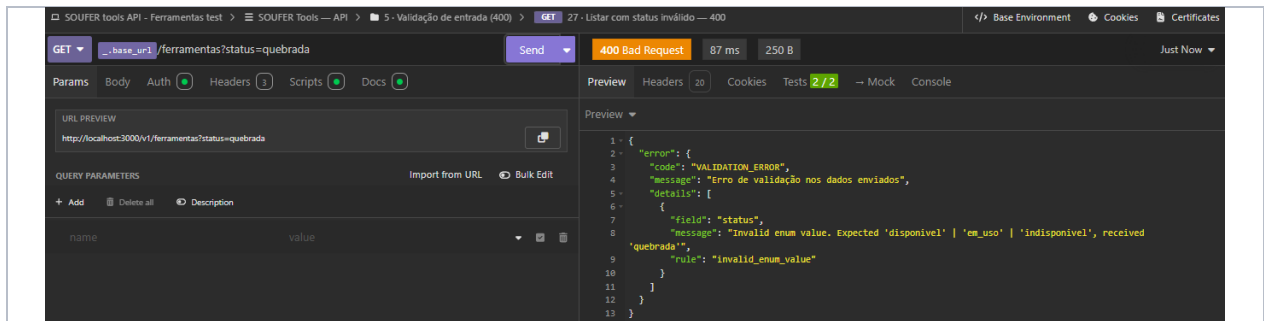
Requisição 27 · Listar com status inválido (400)

O que faz. Filtra por um status que não existe (*quebrada*).

Por que existe. Os únicos status válidos são *disponivel*, *em_uso* e *indisponivel* (regra de negócio nº 2 do projeto).

Método e rota	GET /v1/ferramentas
Query	status=quebrada
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 400 <i>VALIDATION_ERROR</i> .

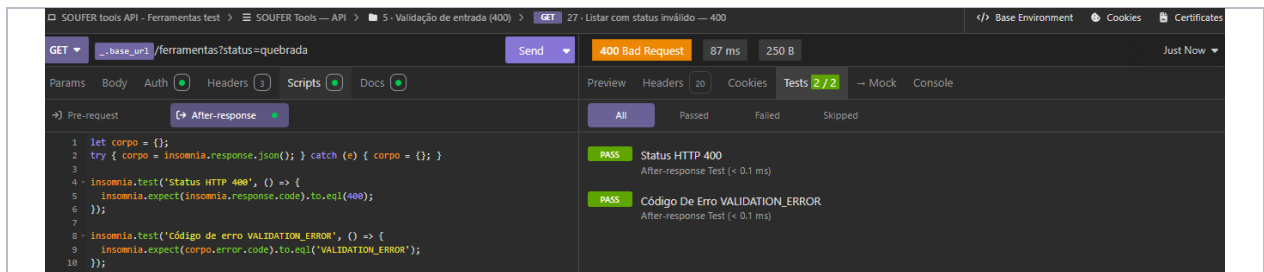
Print 1 · Requisição e resposta. Requisição *GET /ferramentas?status=quebrada* (aba Params). A resposta é *400 Bad Request* (87 ms) com *VALIDATION_ERROR*; o campo *status* recebe “Invalid enum value. Expected ‘disponivel’ | ‘em_uso’ | ‘indisponivel’, received ‘quebrada’” (regra *invalid_enum_value*), listando os três únicos status válidos.



The screenshot shows the Insomnia client interface. The URL is `http://localhost:3000/v1/ferramentas?status=quebrada`. The response is a *400 Bad Request* with a status of 87 ms and a body of 250 B. The response body is a JSON object:

```
{  "error": {    "code": "VALIDATION_ERROR",    "message": "Erro de validação nos dados enviados",    "details": {      "field": "status",      "message": "Invalid enum value. Expected 'disponivel' | 'em_uso' | 'indisponivel', received 'quebrada'",      "rule": "invalid_enum_value"    }  } }
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *VALIDATION_ERROR*”.



The screenshot shows the Insomnia client interface with the Scripts and Tests tabs. The Scripts tab shows an after-response script:

```
1 let corpo = {};  
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }  
3  
4 - insomnia.test('Status HTTP 400', () => {  
5   insomnia.expect(insomnia.response.code).to.eql(400);  
6 });  
7  
8 - insomnia.test('Código de erro VALIDATION_ERROR', () => {  
9   insomnia.expect(corpo.error.code).to.eql('VALIDATION_ERROR');  
10 });
```

The Tests tab shows two tests, both of which passed:

- PASS** Status HTTP 400
After-response Test (< 0.1 ms)
- PASS** Código De Erro *VALIDATION_ERROR*
After-response Test (< 0.1 ms)

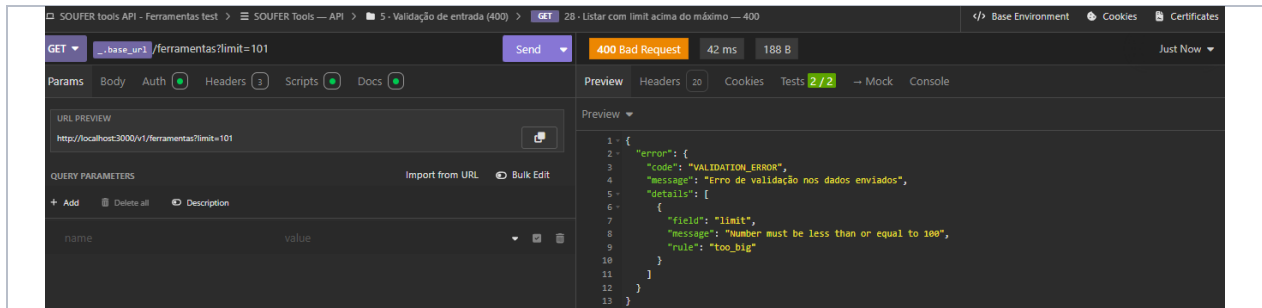
Requisição 28 · Listar com limit acima do máximo (400)

O que faz. Pede 101 itens por página.

Por que existe. O máximo é 100 por página, para proteger a API de consultas gigantes.

Método e rota	GET /v1/ferramentas
Query	Limit=101
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 400 <i>VALIDATION_ERROR</i> .

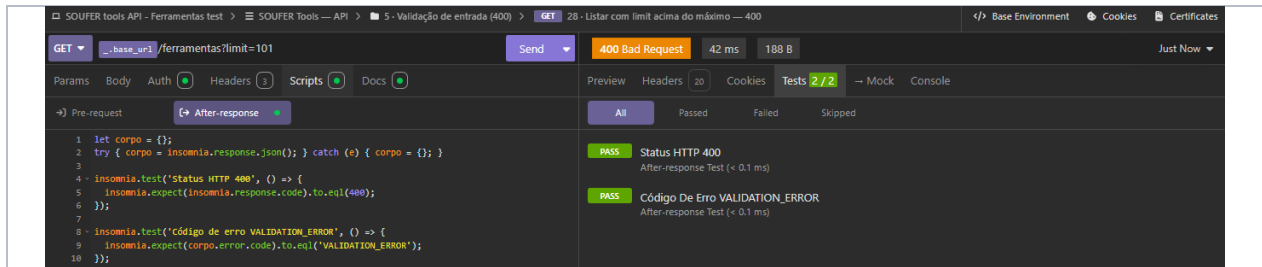
Print 1 · Requisição e resposta. Requisição *GET /ferramentas?Limit=101*. A resposta é *400 Bad Request* (42 ms) com *VALIDATION_ERROR*; o campo *Limit* recebe “Number must be less than or equal to 100” (regra *too_big*). O máximo é 100 itens por página.



The screenshot shows the Insomnia API client interface. The URL is `http://localhost:3000/v1/ferramentas?limit=101`. The response is a **400 Bad Request** (42 ms, 188 B). The response body is a JSON object:

```
1: {
2:   "error": {
3:     "code": "VALIDATION_ERROR",
4:     "message": "Erro de validação nos dados enviados",
5:     "details": [
6:       {
7:         "field": "limit",
8:         "message": "Number must be less than or equal to 100",
9:         "rule": "too_big"
10:      }
11:     ]
12:   }
13: }
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *VALIDATION_ERROR*”.



The screenshot shows the Insomnia API client interface with the **Scripts** and **Tests** tabs selected. The **Scripts** tab shows the following JavaScript code:

```
1 let corpo = {};
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }
3
4 insomnia.test("Status HTTP 400", () => {
5   insomnia.expect(insomnia.response.code).to.eql(400);
6 });
7
8 insomnia.test("Código de erro VALIDATION_ERROR", () => {
9   insomnia.expect(corpo.error.code).to.eql("VALIDATION_ERROR");
10 });
```

The **Tests** tab shows two tests that passed:

- PASS** Status HTTP 400 (After-response Test (< 0.1 ms))
- PASS** Código De Erro VALIDATION_ERROR (After-response Test (< 0.1 ms))

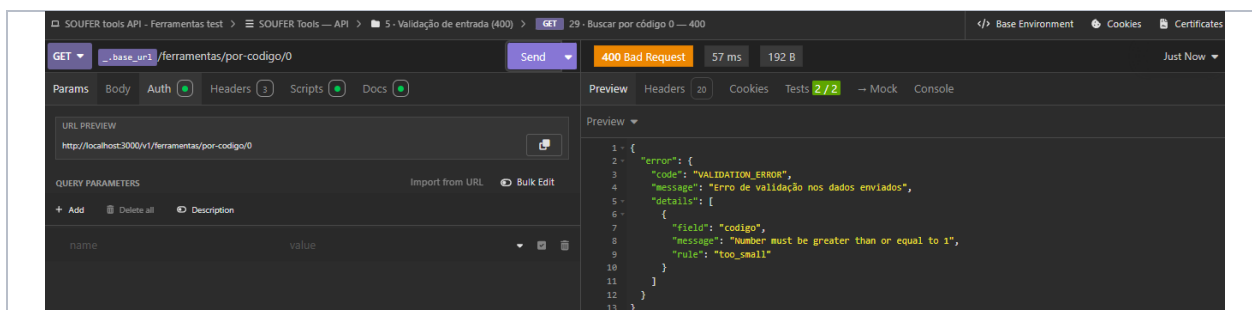
Requisição 29 · Buscar por código 0 (400)

O que faz. Consulta pelo código 0.

Por que existe. O código de identificação vai de 1 a 9999. Este é o limite inferior.

Método e rota	GET /v1/ferramentas/por-codigo/0
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 400 <i>VALIDATION_ERROR</i> .

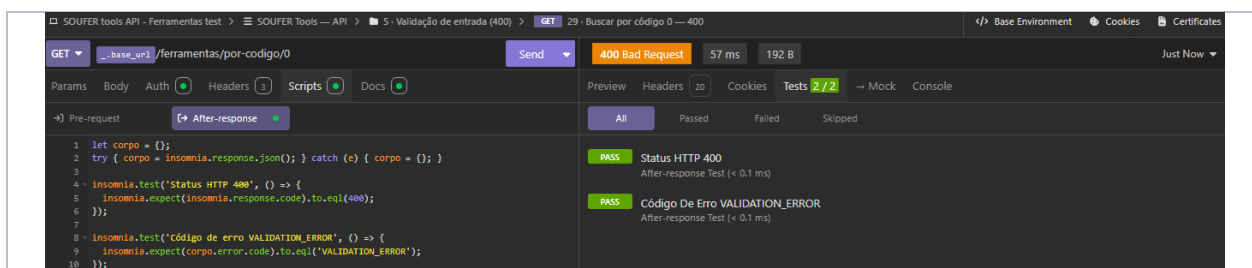
Print 1 · Requisição e resposta. Requisição *GET /ferramentas/por-codigo/0*. A resposta é **400 Bad Request** (57 ms) com *VALIDATION_ERROR*; o campo *codigo* recebe “Number must be greater than or equal to 1” (regra *too_small*), o limite inferior do código.



The screenshot shows the Insomnia interface for a GET request to `/ferramentas/por-codigo/0`. The response is a 400 Bad Request with a status of 57 ms and 192 B. The response body is a JSON object:

```
{  "error": {    "code": "VALIDATION_ERROR",    "message": "Erro de validação nos dados enviados",    "details": {      "field": "codigo",      "message": "Number must be greater than or equal to 1",      "rule": "too_small"    }  }}
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *VALIDATION_ERROR*”.



The screenshot shows the Insomnia interface for the same GET request. The Scripts tab is active, showing a script that validates the response. The Tests tab is also active, showing two tests that passed:

- PASS** Status HTTP 400 (After-response Test (< 0.1 ms))
- PASS** Código De Erro *VALIDATION_ERROR* (After-response Test (< 0.1 ms))

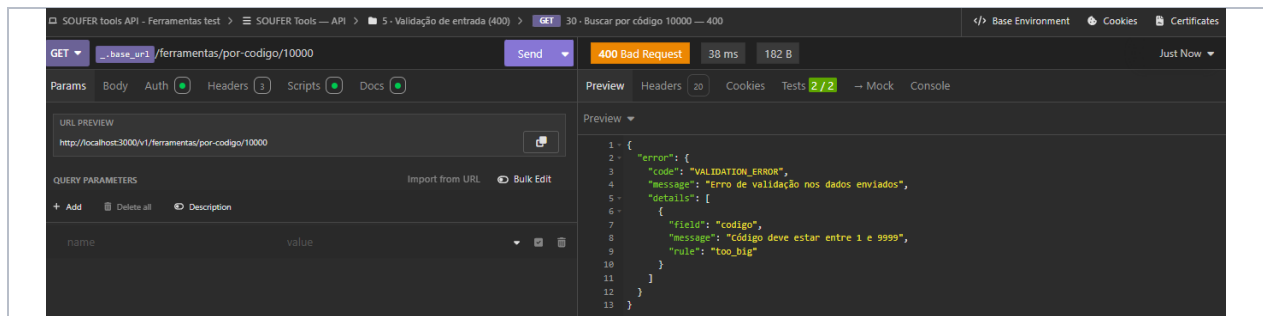
Requisição 30 · Buscar por código 10000 (400)

O que faz. Consulta pelo código 10000.

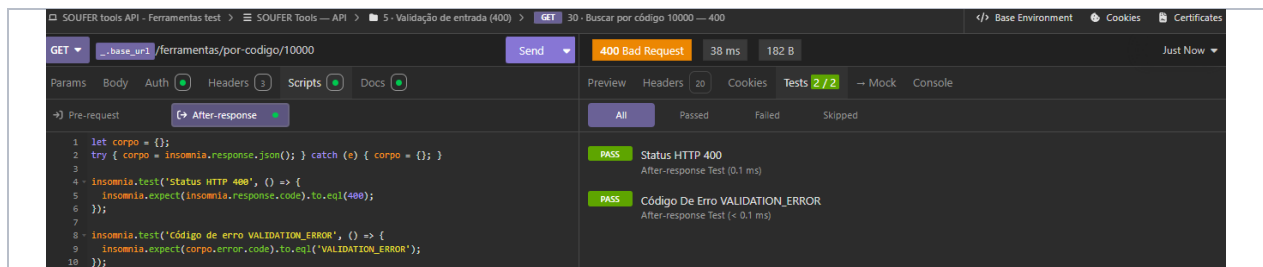
Por que existe. Limite superior: acima de 9999 o código é inválido (coluna SMALLINT com 4 dígitos).

Método e rota	GET /v1/ferramentas/por-codigo/10000
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 400 <i>VALIDATION_ERROR</i> .

Print 1 · Requisição e resposta. Requisição *GET /ferramentas/por-codigo/10000*. A resposta é *400 Bad Request* (38 ms) com *VALIDATION_ERROR*; o campo *codigo* recebe “Código deve estar entre 1 e 9999” (regra *too_big*), o limite superior do código.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 400”; “Código de erro *VALIDATION_ERROR*”.



6.6 Recurso inexistente (404)

Usam um id que não existe (999999) para provar que cada rota responde HTTP 404 *FERRAMENTA_NOT_FOUND*. Nenhuma altera o banco.

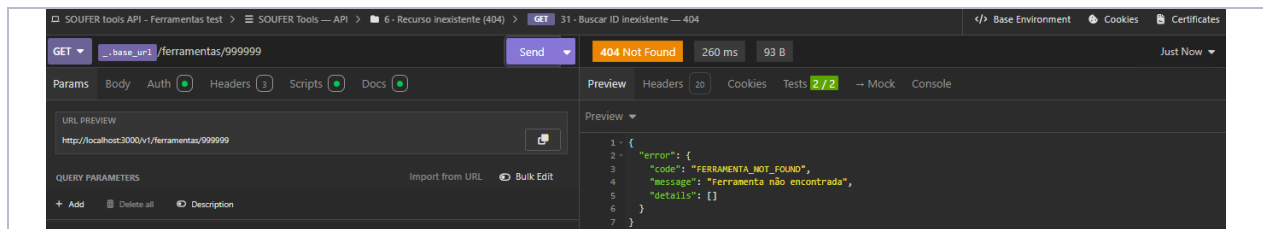
Requisição 31 · Buscar ID inexistente (404)

O que faz. Busca a ferramenta de id 999999.

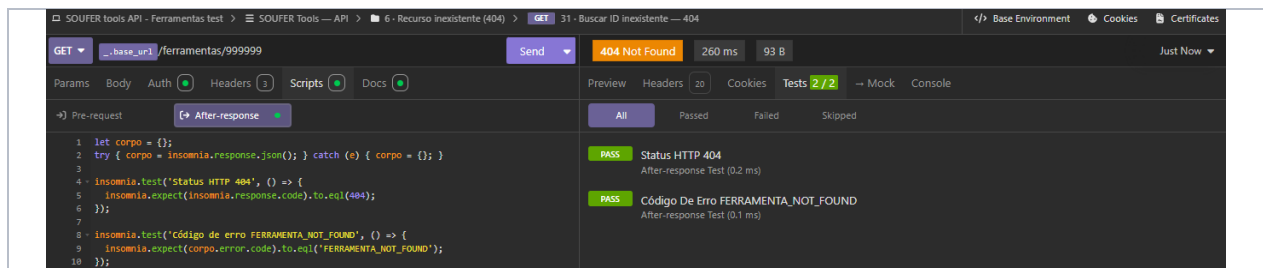
Por que existe. Garante 404 (e não 200 vazio ou erro 500) quando o registro não existe.

Método e rota	GET /v1/ferramentas/999999
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 404 <i>FERRAMENTA_NOT_FOUND</i> .

Print 1 · Requisição e resposta. Requisição *GET /ferramentas/999999*. A resposta é **404 Not Found** (260 ms, 93 B) com o envelope de erro *FERRAMENTA_NOT_FOUND* e a mensagem “Ferramenta não encontrada”, sem nenhum dado.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 404”; “Código de erro *FERRAMENTA_NOT_FOUND*”.



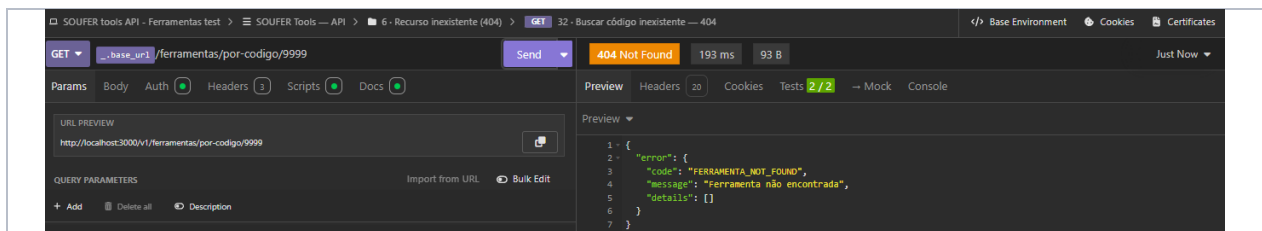
Requisição 32 · Buscar código inexistente (404)

O que faz. Busca pelo código 9999.

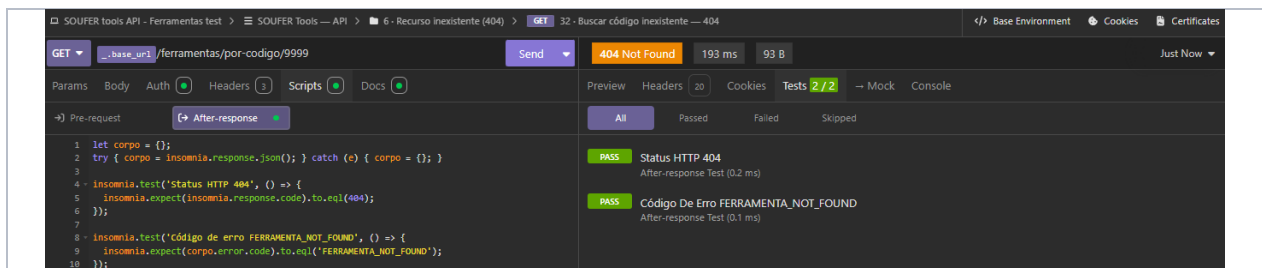
Por que existe. Código válido no formato, mas sem ferramenta ativa com esse número.

Método e rota	GET /v1/ferramentas/por-codigo/9999
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 404 <i>FERRAMENTA_NOT_FOUND</i> .

Print 1 · Requisição e resposta. Requisição *GET /ferramentas/por-codigo/9999*, um código válido em formato, mas sem ferramenta ativa. A resposta é **404 Not Found** (193 ms) com *FERRAMENTA_NOT_FOUND*.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 404”; “Código de erro *FERRAMENTA_NOT_FOUND*”.



Observações. O seed cria 50 ferramentas, então o código 9999 não é usado. Se alguém o atribuir, o teste deixa de valer.

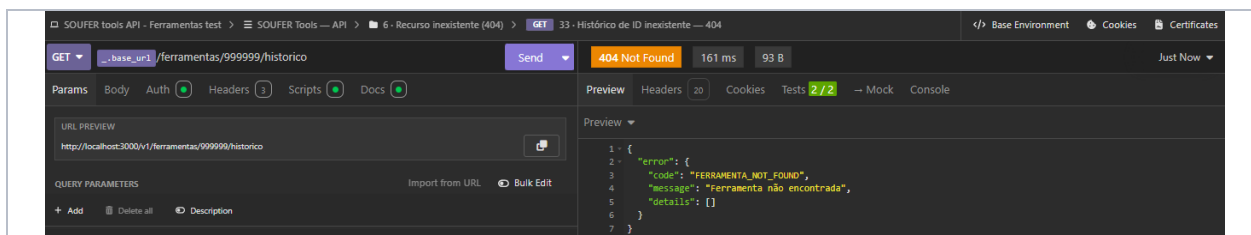
Requisição 33 · Histórico de ID inexistente (404)

O que faz. Consulta o histórico de uma ferramenta inexistente.

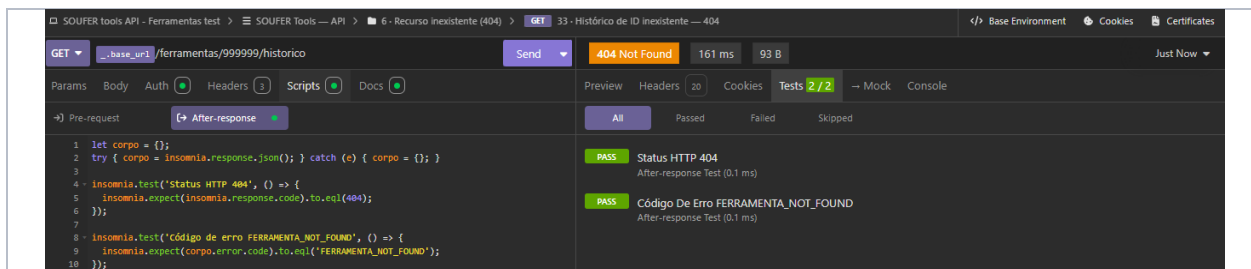
Por que existe. Cobre o 404 da rota de histórico.

Método e rota	GET /v1/ferramentas/999999/historico
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 404 <i>FERRAMENTA_NOT_FOUND</i> .

Print 1 · Requisição e resposta. Requisição *GET /ferramentas/999999/historico*. A resposta é *404 Not Found* (161 ms) com *FERRAMENTA_NOT_FOUND*: a rota de histórico também não inventa dados para uma ferramenta que não existe.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 404”; “Código de erro *FERRAMENTA_NOT_FOUND*”.



Requisição 34 · Atualizar ID inexistente (404)

O que faz. Tenta atualizar uma ferramenta inexistente.

Por que existe. Garante que a atualização não cria registro nem falha silenciosamente.

Método e rota	PATCH /v1/ferramentas/999999
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Corpo (JSON)	{ "marca": "X" }
Resultado esperado	HTTP 404 <i>FERRAMENTA_NOT_FOUND</i> .

Print 1 · Requisição e resposta. Requisição *PATCH /ferramentas/999999* com corpo válido (*{ "marca": "X" }*), para que o único erro possível seja o id. A resposta é **404 Not Found** (166 ms) com *FERRAMENTA_NOT_FOUND*.

The screenshot shows the Insomnia client interface. The request is a PATCH to `/ferramentas/999999` with a JSON body `{ "marca": "X" }`. The response is a 404 Not Found with a status of 166 ms and 93 B. The response body is a JSON object: `{ "error": { "code": "FERRAMENTA_NOT_FOUND", "message": "ferramenta não encontrada", "details": [] } }`.

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 404”; “Código de erro *FERRAMENTA_NOT_FOUND*”.

The screenshot shows the Insomnia client interface with the 'After-response' script and the 'Tests' tab. The script is as follows:

```
1 let corpo = {};  
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }  
3  
4 - insomnia.test('Status HTTP 404', () => {  
5   insomnia.expect(insomnia.response.code).toEqual(404);  
6 });  
7  
8 - insomnia.test('Código de erro FERRAMENTA_NOT_FOUND', () => {  
9   insomnia.expect(corpo.error.code).toEqual('FERRAMENTA_NOT_FOUND');  
10 });
```

The tests tab shows two passed tests:

- PASS** Status HTTP 404
After-response Test (< 0.1 ms)
- PASS** Código De Erro FERRAMENTA_NOT_FOUND
After-response Test (< 0.1 ms)

Requisição 35 · Marcar etiqueta de ID inexistente (404)

O que faz. Marca a etiqueta como impressa numa ferramenta inexistente.

Por que existe. Cobre o 404 da rota *etiqueta-impressa*.

Método e rota	<code>PATCH /v1/ferramentas/999999/etiqueta-impressa</code>
Autenticação	Bearer Token = variável <code>token</code> (almoxarife)
Resultado esperado	HTTP 404 <code>FERRAMENTA_NOT_FOUND</code> .

Print 1 · Requisição e resposta. Requisição `PATCH /ferramentas/999999/etiqueta-impressa`. A resposta é `404 Not Found` (251 ms) com `FERRAMENTA_NOT_FOUND`.

The screenshot shows the Insomnia client interface. The top bar indicates the request method is `PATCH` and the URL is `base_url /ferramentas/999999/etiqueta-impressa`. The response status is `404 Not Found` with a response time of 251 ms and a body size of 93 B. The response body is displayed in the Preview tab, showing a JSON object:

```
{  "error": {    "code": "FERRAMENTA_NOT_FOUND",    "message": "ferramenta não encontrada",    "details": []  }}
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 404”; “Código de erro `FERRAMENTA_NOT_FOUND`”.

The screenshot shows the Insomnia client interface with the `After-response` script and the `Tests` tab. The script is as follows:

```
1 let corpo = {};  
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }  
3  
4 - insomnia.test('Status HTTP 404', () => {  
5   insomnia.expect(insomnia.response.code).to.eql(404);  
6 });  
7  
8 - insomnia.test('Código de erro FERRAMENTA_NOT_FOUND', () => {  
9   insomnia.expect(corpo.error.code).to.eql('FERRAMENTA_NOT_FOUND');  
10 });
```


The `Tests` tab shows two tests that passed:
1. `PASS Status HTTP 404` (After-response Test (0.1 ms))
2. `PASS Código De Erro FERRAMENTA_NOT_FOUND` (After-response Test (< 0.1 ms))

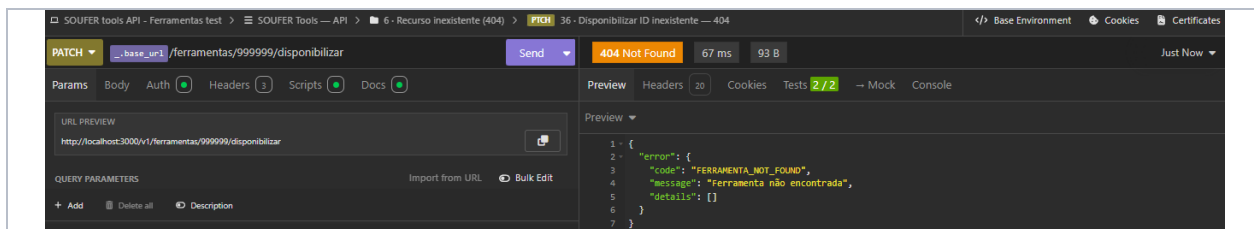
Requisição 36 · Disponibilizar ID inexistente (404)

O que faz. Tenta disponibilizar uma ferramenta inexistente.

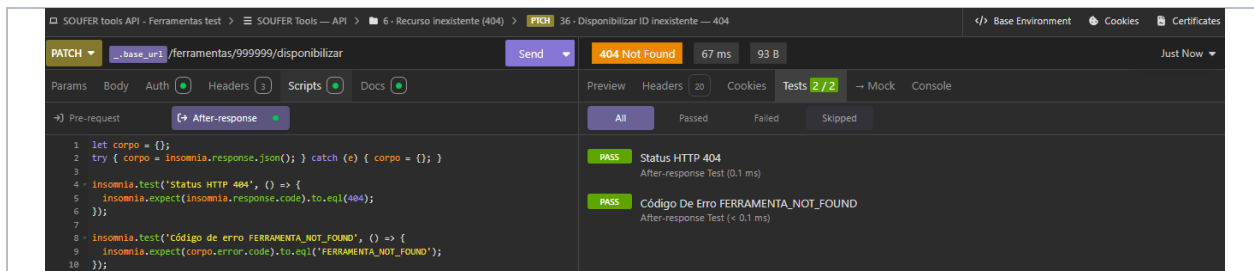
Por que existe. Cobre o 404 da rota *disponibilizar*.

Método e rota	<code>PATCH /v1/ferramentas/999999/disponibilizar</code>
Autenticação	Bearer Token = variável <code>token</code> (almoxarife)
Resultado esperado	HTTP 404 <code>FERRAMENTA_NOT_FOUND</code> .

Print 1 · Requisição e resposta. Requisição `PATCH /ferramentas/999999/disponibilizar`. A resposta é `404 Not Found` (67 ms) com `FERRAMENTA_NOT_FOUND`.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 404”; “Código de erro `FERRAMENTA_NOT_FOUND`”.



6.7 Encerramento — baixa lógica (DELETE)

Remove (baixa lógica) a ferramenta criada na pasta 2 e confirma que ela deixa de existir para a API. É o "D" do CRUD e fica por último porque as pastas anteriores usam essa ferramenta.

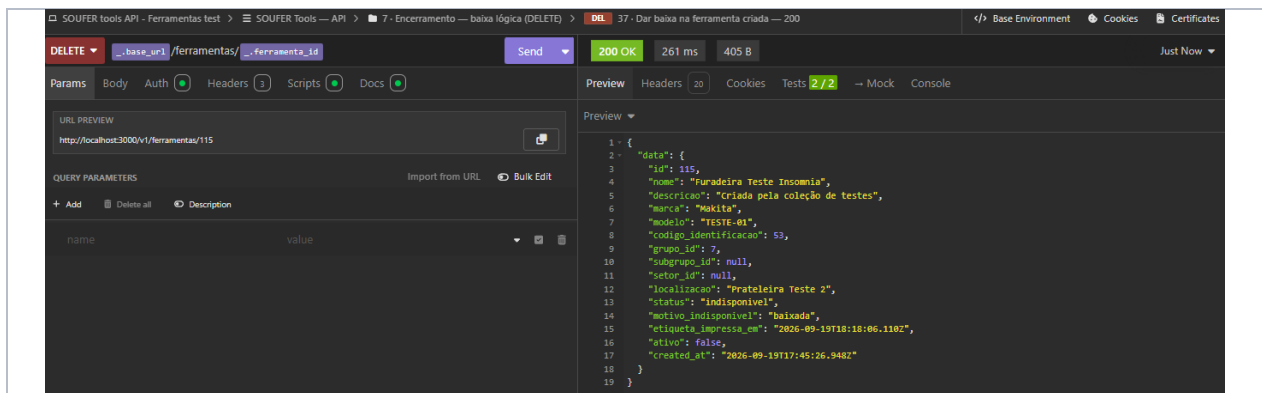
Requisição 37 · Dar baixa na ferramenta criada (200)

O que faz. Dá baixa na ferramenta de teste.

Por que existe. Teste do "D" (Delete). A baixa é lógica: o registro não é apagado, apenas marcado como *ativo = false*, status *indisponivel* e motivo *baixada*, preservando o histórico.

Método e rota	<i>DELETE</i> /v1/ferramentas/:id
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 200 com a ferramenta baixada (<i>ativo = false</i>).

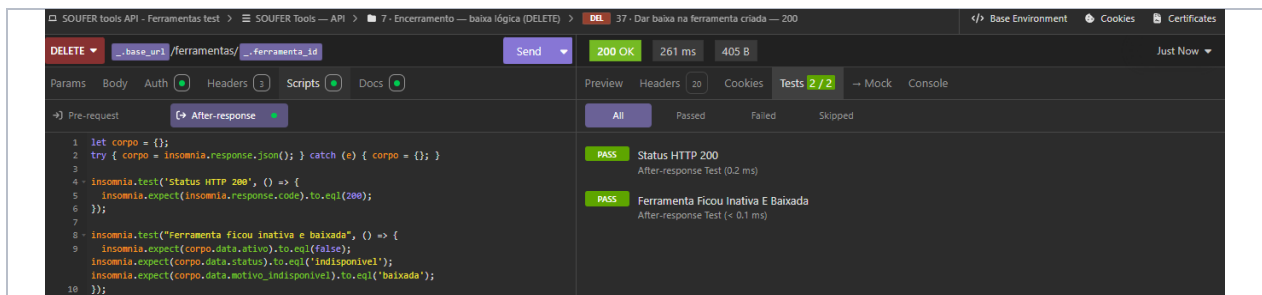
Print 1 · Requisição e resposta. Requisição *DELETE* .../ferramentas/115 (a ferramenta criada na requisição 04). A resposta *200 OK* (261 ms) mostra a baixa lógica: *ativo* false, *status* "indisponivel" e *motivo_indisponivel* "baixada". O registro continua no banco, com o histórico preservado.



The screenshot shows the Insomnia client interface. The top bar indicates the request method is **DELETE** and the URL is `/v1/ferramentas/:id`. The response status is **200 OK** with a response time of 261 ms and a body size of 405 B. The response body is a JSON object:

```
1 {
2   "data": {
3     "id": 115,
4     "nome": "Furadeira Teste Insomnia",
5     "descricao": "Criada pela coleção de testes",
6     "marca": "Makita",
7     "modelo": "TESTE-01",
8     "codigo_identificacao": 53,
9     "grupo_id": 7,
10    "subgrupo_id": null,
11    "setor_id": null,
12    "localizacao": "Prateleira Teste 2",
13    "status": "Indisponivel",
14    "motivo_indisponivel": "baixada",
15    "etiqueta_impressa_em": "2026-09-19T18:18:06.118Z",
16    "ativo": false,
17    "created_at": "2026-09-19T17:45:26.948Z"
18  }
19 }
```

Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): "Status HTTP 200"; "Ferramenta ficou inativa e baixada".



The screenshot shows the Insomnia client interface with the **Scripts** and **Tests** tabs active. The **Scripts** tab shows the following JavaScript code:

```
1 let corpo = {};
2 try { corpo = insomnia.response.json(); } catch (e) { corpo = {}; }
3
4 - insomnia.test('Status HTTP 200', () => {
5   insomnia.expect(insomnia.response.code).to.eql(200);
6 });
7
8 - insomnia.test('Ferramenta ficou inativa e baixada', () => {
9   insomnia.expect(corpo.data.ativo).to.eql(false);
10  insomnia.expect(corpo.data.status).to.eql('indisponivel');
11  insomnia.expect(corpo.data.motivo_indisponivel).to.eql('baixada');
12 });
```

The **Tests** tab shows two tests that passed:

- PASS** Status HTTP 200 (After-response Test (0.2 ms))
- PASS** Ferramenta Ficou Inativa E Baixada (After-response Test (< 0.1 ms))

Observações. Convenção REST: um *DELETE* costuma responder 204 sem corpo. A API atual devolve 200 com o registro baixado; a coleção segue o comportamento real. Ver "Convenções REST" no tutorial.

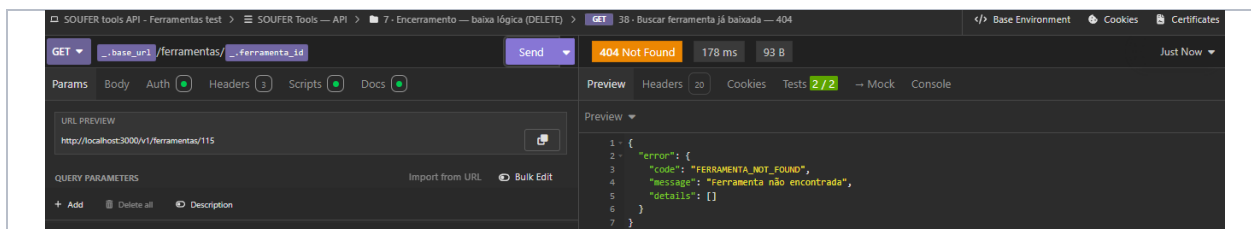
Requisição 38 · Buscar ferramenta já baixada (404)

O que faz. Busca de novo a ferramenta que acabou de ser baixada.

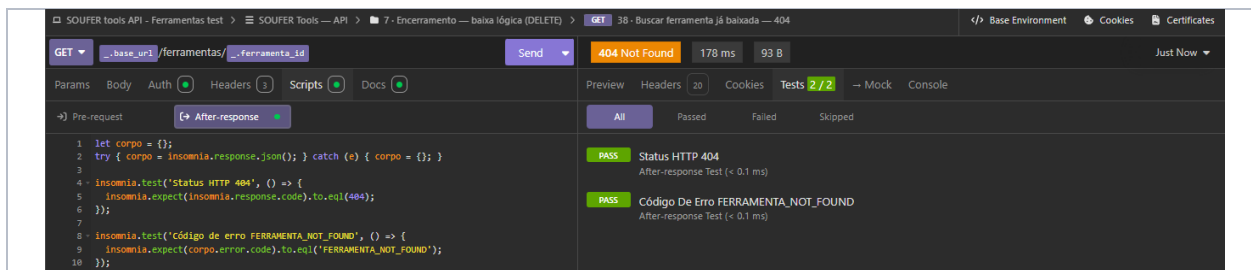
Por que existe. Prova que a baixa surtiu efeito: ferramentas inativas não aparecem mais nas consultas.

Método e rota	GET /v1/ferramentas/:id
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 404 <i>FERRAMENTA_NOT_FOUND</i> .

Print 1 · Requisição e resposta. Requisição *GET .../ferramentas/115*, a mesma ferramenta, agora baixada. A resposta é *404 Not Found* (178 ms) com *FERRAMENTA_NOT_FOUND*: a baixa surtiu efeito e ferramentas inativas não aparecem mais nas consultas.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 404”; “Código de erro *FERRAMENTA_NOT_FOUND*”.



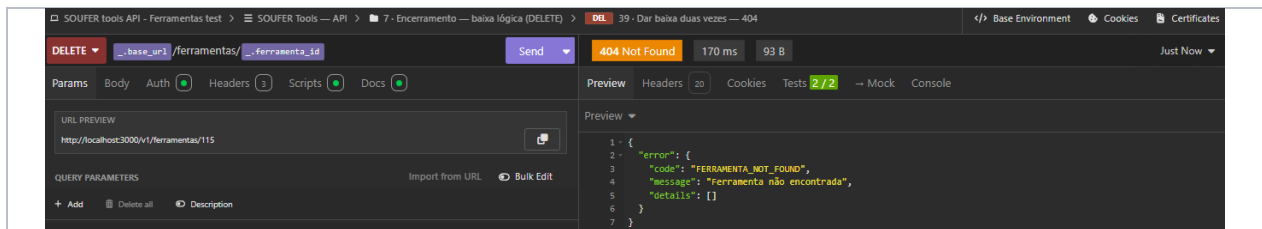
Requisição 39 · Dar baixa duas vezes (404)

O que faz. Repete a baixa na mesma ferramenta.

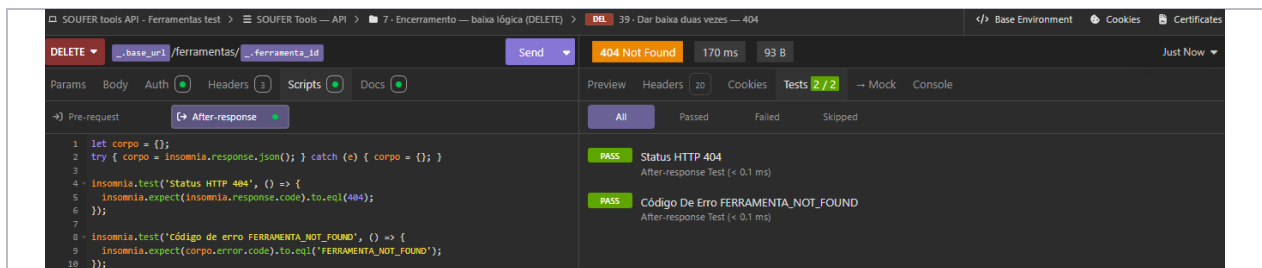
Por que existe. Verifica o comportamento em repetição: a segunda chamada não deve corromper nada e informa que a ferramenta não existe mais.

Método e rota	<i>DELETE /v1/ferramentas/:id</i>
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 404 <i>FERRAMENTA_NOT_FOUND</i> .

Print 1 · Requisição e resposta. Requisição *DELETE .../ferramentas/115* repetida. A resposta é *404 Not Found* (170 ms) com *FERRAMENTA_NOT_FOUND*: a segunda baixa não altera nada e informa que a ferramenta não existe mais.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 404”; “Código de erro *FERRAMENTA_NOT_FOUND*”.



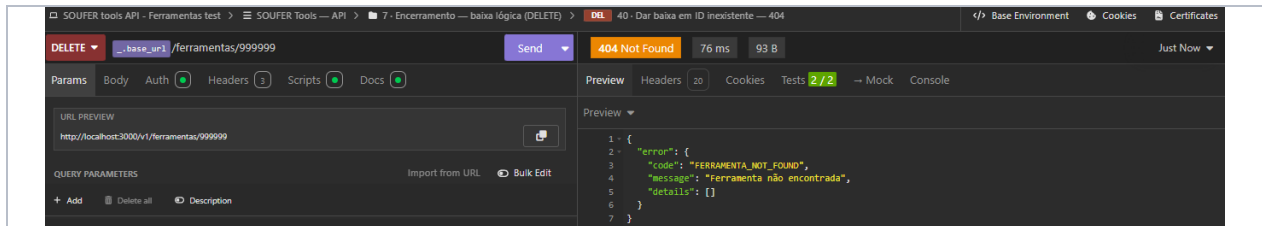
Requisição 40 · Dar baixa em ID inexistente (404)

O que faz. Tenta dar baixa numa ferramenta que nunca existiu.

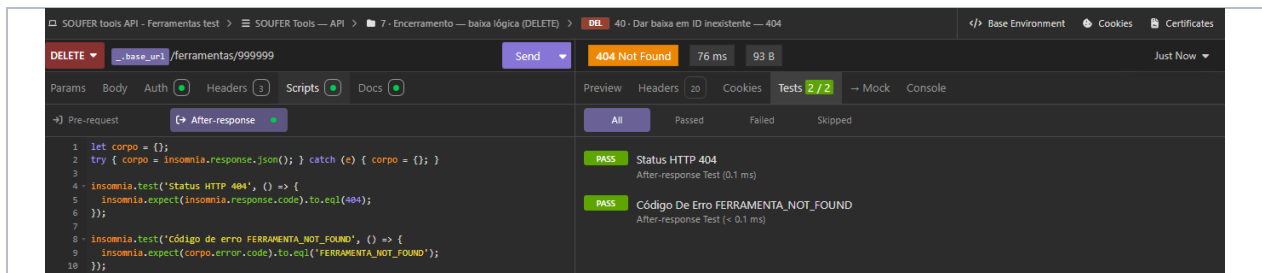
Por que existe. Cobre o 404 do *DELETE*.

Método e rota	<i>DELETE</i> /v1/ferramentas/999999
Autenticação	Bearer Token = variável <i>token</i> (almoxarife)
Resultado esperado	HTTP 404 <i>FERRAMENTA_NOT_FOUND</i> .

Print 1 · Requisição e resposta. Requisição *DELETE* /*ferramentas/999999*, um id que nunca existiu. A resposta é **404 Not Found** (76 ms) com *FERRAMENTA_NOT_FOUND*.



Print 2 · Script e testes automáticos. Aba Scripts (After-response) com o script que valida a resposta, e aba Tests com 2/2 testes aprovados (PASS): “Status HTTP 404”; “Código de erro *FERRAMENTA_NOT_FOUND*”.



7. Resultado

O CRUD de ferramentas está implementado e testado de ponta a ponta no Insomnia: 40 requisições (15 de sucesso ou de apoio e 25 de erro ou regra de negócio) e 85 testes automáticos aprovados. Os testes cobrem autenticação e autorização (401 e 403), validação de entrada (400), recurso inexistente (404), conflitos de regra de negócio (409) e o ciclo completo de cadastrar, consultar, atualizar, marcar etiqueta, disponibilizar e dar baixa.

Durante a preparação foi identificado e corrigido um ponto de convenção: a atualização parcial de ferramentas passou de *PUT* para *PATCH*. A coleção do Insomnia, os testes e o tutorial ficaram versionados no repositório, e qualquer integrante consegue repetir os testes importando a coleção e seguindo o tutorial.